Community Experience Distilled

NuGet 2 Essentials

Take advantage of the NuGet Package Manager, both as a consumer and a publisher of .NET class libraries

Foreword by Howard Dierking, Senior Program Manager, Microsoft Corporation

Damir Arh **Dejan Dakić**

[PACKT] PUBLISHING open source* community experience distilled

www.allitebooks.com

NuGet 2 Essentials

Take advantage of the NuGet Package Manager, both as a consumer and a publisher of .NET class libraries

Damir Arh

Dejan Dakić

[PACKT] PUBLISHING open source community experience distilled

BIRMINGHAM - MUMBAI

www.allitebooks.com

NuGet 2 Essentials

Copyright © 2013 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the authors, nor Packt Publishing, and its dealers and distributors will be held liable for any damages caused or alleged to be caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

First published: November 2013

Production Reference: 1131113

Published by Packt Publishing Ltd.
Livery Place
35 Livery Street
Birmingham B3 2PB, UK.

ISBN 978-1-78216-586-6

www.packtpub.com

Cover Image by Artie Ng (artherng@yahoo.com.au)

www.allitebooks.com

Credits

Authors
Damir Arh
Dejan Dakić

Reviewers
Alvin Ashcraft
John Gozde
Joseph Guadagno

Acquisition Editors
Nikhil Karkal
Mary Nadar

Commissioning Editor
Priyanka Shah

Technical Editor
Vivek Pillai

Copy Editors
Roshni Banerjee
Insiya Morbiwala

Project Coordinator
Sageer Parkar

Proofreader
Sandra Hopper

Indexer
Rekha Nair

Production Coordinator
Arvindkumar Gupta

Cover Work
Arvindkumar Gupta

www.allitebooks.com

www.allitebooks.com

Foreword

I began my career in software development back in the days when dynamic web sites were created with Perl scripts (or lower-level CGI libraries for the adventurous) and desktop applications were created with C++ or Visual Basic 4. These programming environments, while enabling the development of richer and more powerful applications, required a lot from the developer. The most common shared functionality was provided by large platform SDKs and frameworks; however, the developer was still left to write common functions in each of his or her projects. The strategy for reusing these smaller bits of functionality was, at that time, copying and pasting.

As time progressed, development on the Microsoft platform grew more componentized with technologies such as COM, and later on .NET Framework assemblies. However, each of these improvements still had a few issues that ultimately prevented their widespread adoption as a general vehicle for component sharing.

There was no standard means for packing and installing components. Strategies ranged from making binaries available via file share or FTP to complete executables or MSIs.

There was no standard means of discovering and delivering components. Your success in reusing a component was a function of your search engine or personal network of colleagues.

Most component technologies modified the developer's machine state. This meant that two developers needed to have set up their machines the same way in order to work on the same project.

The result of these characteristics was that most component re-use was in the form of purchasing large component libraries from known vendors who had the ability to invest in creating the installation and delivery infrastructure as well to market their libraries to developers and software companies.

www.allitebooks.com

As you'll read in this book, the single greatest thing about NuGet is that it democratizes the process of sharing components. NuGet makes it easy for a developer or group of developers to create a shareable unit of functionality and publish it to a known location, making it easily discoverable by anyone who may then be able to benefit from that functionality.

I joined the NuGet team nearly two years ago, just before we released NuGet 1.6. At that point, NuGet was just introducing features such as package restore and support for semantic versioning prerelease tokens. Suffice to say, NuGet has evolved considerably since those days, with features such as automatic package restore, support for C++ projects, and enterprise features such as hierarchical configuration—along with a litany of additional features and fixes.

Given the pace of NuGet innovation, one of the things that makes this book valuable is its accessibility, regardless of the reader's familiarity with NuGet. If you are new to NuGet, you can start at the beginning to get an understanding of the big picture and then work through the more detailed aspects of package management with NuGet. If you are already familiar with NuGet and package management and want to learn more about a specific aspect of NuGet, the book is organized in such a manner as to make it easy to jump directly to the relevant section.

The second aspect of the book that I appreciate is its task-focused design. The authors provide enough contexts to be helpful, but then quickly move into walking the reader through actually using NuGet in the way that they described. Given that all of us are trying to get more results from our already time-constrained lives, I greatly appreciate this "learn by doing" approach. This approach is also quite consistent with the spirit of NuGet, which is to remove as much friction as possible when it comes to building great software projects.

My hope is that after reading this book, you'll have a better sense of how to use NuGet to improve the development workflows in your own projects—maybe in ways that you had not previously thought about. I also hope that for many of you, this book piques your interest in the NuGet open source project and serves as a starting point for your deeper involvement as a contributor to the future of NuGet.

Howard Dierking
Senior Program Manager, Microsoft Corporation

www.allitebooks.com

About the Authors

Damir Arh has been working with the .NET Framework since it was first released when he was still in university. He has a lot of experience in desktop-based applications, be it Windows Forms or WPF. But he has also done some work in ASP.NET, both web forms and MVC. Lately, he has given more attention to mobile platforms, mostly Windows Store apps and Windows Phone. He's a fan of portable class libraries since he first tried them out.

He has been working on large software projects for over 10 years, gaining experience in source control, continuous integration, and good development practices and patterns to cope with their complexities. He likes to share his knowledge with other developers through his blog at `www.damirscorner.com`, by answering questions at Stack Overflow, and by speaking at local user group meetings and technical conferences. He has been awarded the Microsoft Most Valuable Professional (MVP) for the second year in a row.

Currently, he works as a Software Architect for Adacta, a company specializing in development of software for insurance companies, based in Ljubljana, Slovenia.

First, I'd like to thank Dejan for deciding to join me in the interesting and challenging project of writing this book. His help made the amount of work required easier to handle. Next, I want to thank the people at Packt Publishing for originally approaching me with the idea for this book, for giving us both the opportunity to write it, and of course for being there throughout the project and helping us getting it completed.

I'd also like to thank the reviewers and the editors for their valuable feedback; they have helped to make this book better.

Last but not least, I'd like to thank my parents and friends for their support and understanding during the evenings and weekends I spent working on this book. Thank you.

---

www.allitebooks.com

Dejan Dakić has been working with the .NET Framework for over five years. He has the most experience with ASP.NET but has also done a lot of work in Windows Forms and WPF. He has been active in mobile development since Windows Mobile, and now he has progressed to Windows RT. Lately, he has been spending time in the Windows Azure environment, focusing on web development, both client- and server-side.

Currently, he works in a team with Damir as a software developer for Adacta—a company specializing in development of software for insurance companies, which is based in Ljubljana, Slovenia.

First, I'd like to thank Damir for inviting me to this interesting project of writing this book. He has been an admirable mentor throughout the whole process.

I'd also like to thank the people at Packt Publishing, reviewers, and the editors for their valuable feedback.

I'd also like to thank my parents, my sister, and friends for their support, comments, and feedback.

Last but not least, I'd like to thank my girlfriend Pia for supporting me and putting up with my late night works on this project. Thank you.

---

www.allitebooks.com

About the Reviewers

Alvin Ashcraft is a Software Architect living near Philadelphia, PA. He has dedicated his 18-years career to building software with C#, Visual Studio, WPF, ASP.NET MVC, Windows Phone, Entity Framework, and SQL Server. He has been awarded the Microsoft MVP five times, once for Software Architecture in 2009 and the four subsequent years for Visual C#. You can read his daily links for .NET developers on his blog at `alvinashcraft.com`.

He currently works for Genzeon Corporation, developing clinical healthcare software. He has previously been employed with several large software companies, including Allscripts and Oracle. There, he helped create software solutions for financial, business, and healthcare organizations using Microsoft platforms and solutions.

I would like to thank my wonderful wife, Stelene, and our three incredible daughters for their support. They were very understanding when daddy had to do some extra work in the evening to help deliver a useful, high quality NuGet book.

John Gozde is a Canadian software developer who primarily develops rich .NET web applications. At Pandell, his current employer, John guided the company-wide adoption of NuGet for dependency management, which is now an integral part of the overall development environment and infrastructure. When not writing code and evangelizing NuGet, John is either playing drums or riding his bicycle.

www.allitebooks.com

Joseph Guadagno has been in the field of software development for about 20 years or so. He started out with a small book on QuickBASIC, then moved to Visual Basic for DOS, then Visual Basic for Windows, then Visual Basic .NET, and eventually Visual C#—where he has been an MVP for five years. When he is not working at his full time job at QuickenLoan, he donates his time to several community efforts such as INETA North America Board of Directors, the Southeast Valley .NET User Group, and serving on the Parks and Recreation board of the City of Chandler, Arizona. Joe also organizes Desert Code Camp in Chandler, Arizona.

I'd like to thank my lovely wife Deidre, my son JJ, and my daughter Emily for giving me the time to work on the review of this book.

---

www.PacktPub.com

Support files, eBooks, discount offers and more

You might want to visit `www.PacktPub.com` for support files and downloads related to your book.

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at `www.PacktPub.com` and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at `service@packtpub.com` for more details.

At `www.PacktPub.com`, you can also read a collection of free technical articles, sign up for a range of free newsletters and receive exclusive discounts and offers on Packt books and eBooks.

`http://PacktLib.PacktPub.com`

Do you need instant solutions to your IT questions? PacktLib is Packt's online digital book library. Here, you can access, read and search across Packt's entire library of books.

Why Subscribe?

- Fully searchable across every book published by Packt
- Copy and paste, print and bookmark content
- On demand and accessible via web browser

Free Access for Packt account holders

If you have an account with Packt at `www.PacktPub.com`, you can use this to access PacktLib today and view nine entirely free books. Simply use your login credentials for immediate access.

Table of Contents

Preface **1**

Chapter 1: Using a Package Manager in Visual Studio **7**

Benefits of using a package manager **7**

Manually adding third-party library references **8**

Setting up NuGet in Visual Studio **12**

Requirements 12

Installing NuGet 13

Installing from the Visual Studio gallery 13

Installing from the Extension Manager 13

Installing the command-line utility 14

Adding references to third-party libraries using NuGet **14**

Dependencies with NuGet 15

Summary **17**

Chapter 2: Package Management **19**

Updating referenced third-party libraries **19**

Updating an installed package 20

Local package version management 22

Support for different target platforms **23**

Installing a package for an unsupported target platform 25

Using NuGet with source control **26**

Using NuGet on a build server 30

Using Package Manager Console **31**

Summary **33**

Chapter 3: Creating and Publishing a Package **35**

Creating a package with NuGet Package Explorer **35**

Dependencies 40

Publishing the package to the official NuGet Gallery **41**

Creating and publishing a package from the command line **42**
Creating a NuGet specification 42
Packing the package 43
Publishing the package 43
Generating the package during the build process **44**
Understanding package versioning **47**
SemVer 47
The prerelease packages 47
Summary **48**
Chapter 4: Adding Features to a NuGet Package **49**
Package folder structure **49**
Multiplatform support **50**
Testing a NuGet package locally 52
Folder naming conventions 54
Packaging localized satellite assemblies **55**
Adding localized assemblies to the main package 56
Creating satellite packages 58
Adding new files and modifying existing ones **60**
Transforming configuration files 60
The XML-Document-Transform syntax 62
Adding source code files 63
Visual Studio project properties 64
Adding MSBuild targets and properties 65
Including PowerShell scripts in a package **67**
Running a script at install time 68
Adding a PowerShell module 69
Debugging of packages **70**
Creating and publishing a symbol package 70
Consuming a symbol package 72
Summary **75**
Chapter 5: Hosting an Internal NuGet Server **77**
Understanding reasons for hosting NuGet internally **78**
Creating a NuGet package repository **78**
Creating a filesystem-based NuGet repository 79
The NuGet cache 80
Creating a lightweight NuGet server 80
Configuring the NuGet server 81
Accessing packages 83
Using a private feed with package restore 83

The NuGet Gallery 84
Prerequisites 85
Installing the NuGet Gallery 85
Configuring and compiling the NuGet Gallery 85
NuGet server in a continuous integration system 89
Summary 91
Index 93

Preface

Using third-party libraries in .NET projects has been a nontrivial endeavor for quite a long time. Without a centralized catalog and installation process, developers first needed to discover them using their favorite search engine, then installed them manually or using a proprietary tool, and finally referenced them in such a way that the assemblies would be distributed along with the source code. The complexity of this process and the lack of discoverability were a huge roadblock for the expansion of an open source ecosystem. With only a few exceptions, such as NUnit and log4net, developers mostly used the classes that were shipped with the framework.

This started to change in 2010 when the first version of NuGet was released. At that time it was named NuPack, but it was soon renamed to avoid confusion with another already existing software package. NuGet is a package manager tool, similar to Ruby Gems and Linux package management systems. It enables easy discoverability, installation, and upgrading of third-party libraries used in development projects.

In the three years since its release, NuGet has become one of the most important tools for .NET development. It is included with the latest version of Visual Studio, many Microsoft project templates and libraries take advantage of it, and most importantly, it became a standard approach for distribution of third party-libraries, the open source ones in particular. At the time of writing this book, there were over 14,000 packages available.

This book takes the reader through all aspects of using NuGet, from the basics to the advanced scenarios of creating packages and hosting an internal server. It demonstrates all the concepts on real-world examples, making it easier to recognize the opportunities for using NuGet in everyday development work.

What this book covers

Chapter 1, *Using a Package Manager in Visual Studio*, introduces you to NuGet, demonstrates its benefits, and teaches the basics of using it. It will guide you through the NuGet installation process, show you how to reference your first NuGet package, and demonstrate how dependencies between packages are handled.

Chapter 2, *Package Management*, covers advanced aspects of using NuGet in Visual Studio. It starts out with two important aspects of NuGet packages: versioning and support for different target platforms. This is followed by a closer look at how NuGet plays along with source control systems. It ends with an introduction to the NuGet PowerShell console.

Chapter 3, *Creating and Publishing a Package*, focuses on the basics of creating a NuGet package for a library. It shows you how to do it manually and how to automatically create it at the end of every build. It also explains package versioning and leads you through the process of publishing your first package in the NuGet Gallery.

Chapter 4, *Adding Features to a NuGet Package*, drills down into the advanced features of NuGet packages. It describes the contents of a package and gives examples for each of the following features: multiplatform support, multilanguage support, and debugging support. It also explains which types of files can be included in a package and why you would want to include them.

Chapter 5, *Hosting an Internal NuGet Server*, starts with the reasons to host your own NuGet server and explains how to choose the right setup among the three different options, based on your requirements. It includes detailed installation and configuration instructions for setting up any one of them: a file system repository, a simple NuGet server, and your own NuGet gallery.

What you need for this book

The basic requirements for this book are Microsoft Visual Studio 2010 Pro/Premium/Ultimate or any edition of Visual Studio 2012 (including all of the Express editions), Microsoft .NET Framework 4.0 or newer, PowerShell 2.0, and NuGet Package Manager.

Please refer to *Chapter 1*, *Using a Package Manager in Visual Studio*, for detailed information on installation of NuGet and the setup of the environment.

Who this book is for

This book is intended for .NET developers who want to learn more about NuGet. Although it doesn't require any prior knowledge about NuGet itself, it requires familiarity with Visual Studio and the .NET Framework. All the code in the examples are written using C#; therefore, basic knowledge of the language is required as well.

Conventions

In this book, you will find a number of styles of text that distinguish between different kinds of information. Here are some examples of these styles, and an explanation of their meaning.

Code words in text are shown as follows: "To get a list of all commands, type `Get-Help NuGet` in the console."

A block of code is set as follows:

```
var jsonString =
    @"{
        ""title"": ""NuGet 2 Essentials"",
        ""authors"":""Damir Arh & Dejan Dakic"",
        ""publisher"": ""Packt Publishing""
    }";
var parsedJson = Newtonsoft.Json.JsonConvert
                           .DeserializeObject(jsonString);
```

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are set in bold:

```
<?xml version="1.0" encoding="utf-8"?>
<packages>
  <package id="Newtonsoft.Json" version="5.0.1"
    targetFramework="net45" />
</packages>
```

Any command-line input or output is written as follows:

```
PM> Install-Package Newtonsoft.Json
```


www.allitebooks.com

New terms and **important words** are shown in bold. Words that you see on the screen, in menus or dialog boxes for example, appear in the text like this: "Create a new **Console Application** project in Visual Studio."

Warnings or important notes appear in a box like this.

Tips and tricks appear like this.

Reader feedback

Feedback from our readers is always welcome. Let us know what you think about this book—what you liked or may have disliked. Reader feedback is important for us to develop titles that you really get the most out of.

To send us general feedback, simply send an e-mail to `feedback@packtpub.com`, and mention the book title through the subject of your message.

If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, see our author guide on `www.packtpub.com/authors`.

Customer support

Now that you are the proud owner of a Packt book, we have a number of things to help you to get the most from your purchase.

Downloading the example code

You can download the example code files for all Packt books you have purchased from your account at `http://www.packtpub.com`. If you purchased this book elsewhere, you can visit `http://www.packtpub.com/support` and register to have the files e-mailed directly to you.

Errata

Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you find a mistake in one of our books—maybe a mistake in the text or the code—we would be grateful if you would report this to us. By doing so, you can save other readers from frustration and help us improve subsequent versions of this book. If you find any errata, please report them by visiting `http://www.packtpub.com/support`, selecting your book, clicking on the **errata submission form** link, and entering the details of your errata. Once your errata are verified, your submission will be accepted and the errata will be uploaded to our website, or added to any list of existing errata, under the Errata section of that title.

Piracy

Piracy of copyright material on the Internet is an ongoing problem across all media. At Packt, we take the protection of our copyright and licenses very seriously. If you come across any illegal copies of our works, in any form, on the Internet, please provide us with the location address or website name immediately so that we can pursue a remedy.

Please contact us at `copyright@packtpub.com` with a link to the suspected pirated material.

We appreciate your help in protecting our authors, and our ability to bring you valuable content.

Questions

You can contact us at `questions@packtpub.com` if you are having a problem with any aspect of the book, and we will do our best to address it.

1

Using a Package Manager in Visual Studio

NuGet has become one of the best choices for package management in Microsoft .NET. It simplifies handling of third-party libraries, automatically configures your project by adding references to all the necessary assemblies, and updates source and configuration files.

Before NuGet, there was no real dependency management system for .NET projects that could handle adding, saving, and maintaining libraries to solutions and source control systems.

The official NuGet feed is located on `http://www.nuget.org` and it is a place for over 15,000 unique packages.

Benefits of using a package manager

As a project grows, the need for additional functionality increases. Some of this functionality can be found in various libraries, so the first issue depends on the decision whether to use external libraries or not. There is a variety of solutions out there and sometimes software companies try to invent their own. We usually end up with one of the two different approaches when doing work on various projects.

The first approach we encountered is simply not using external libraries. Although not having to deal with libraries saves you all the troubles related to third-party dependencies, but you end up reinventing the wheel over and over again.

The second approach is much better and more common: having all the external assemblies in the root project folder (named as `lib`, `References`, or `Dependencies`) and referencing them in projects where needed. Adding these assemblies—either binaries or source code—in source control can save you the trouble when building a solution or setting up a new development environment.

Once a third-party assembly is added to the repository, it is very likely that the assembly never gets updated to a newer version; we do not receive any notification about the new version being released. Updating to a new library release is complicated. The most common reason for version update is when the newer version solved a specific issue encountered by the development team. Upgrading these libraries can cause headaches, problems with backward compatibility, and even requires migration to a newer version of the .NET Framework.

Software developers often search for these third-party assemblies on various open source code repositories such as GitHub, Codeplex, and similar. Once we find a site for the required library, we will start looking for documentation, date of last build, and last check in; basically, everything that will convince us the library is trustworthy and reliable.

NuGet solves all these problems in a simple and convenient way via a graphical user interface or a PowerShell-based command line named Package Management Console.

In the following sections, we will first demonstrate how we would add a third-party library and then reach the same goal using NuGet. We will also cover the installation of NuGet.

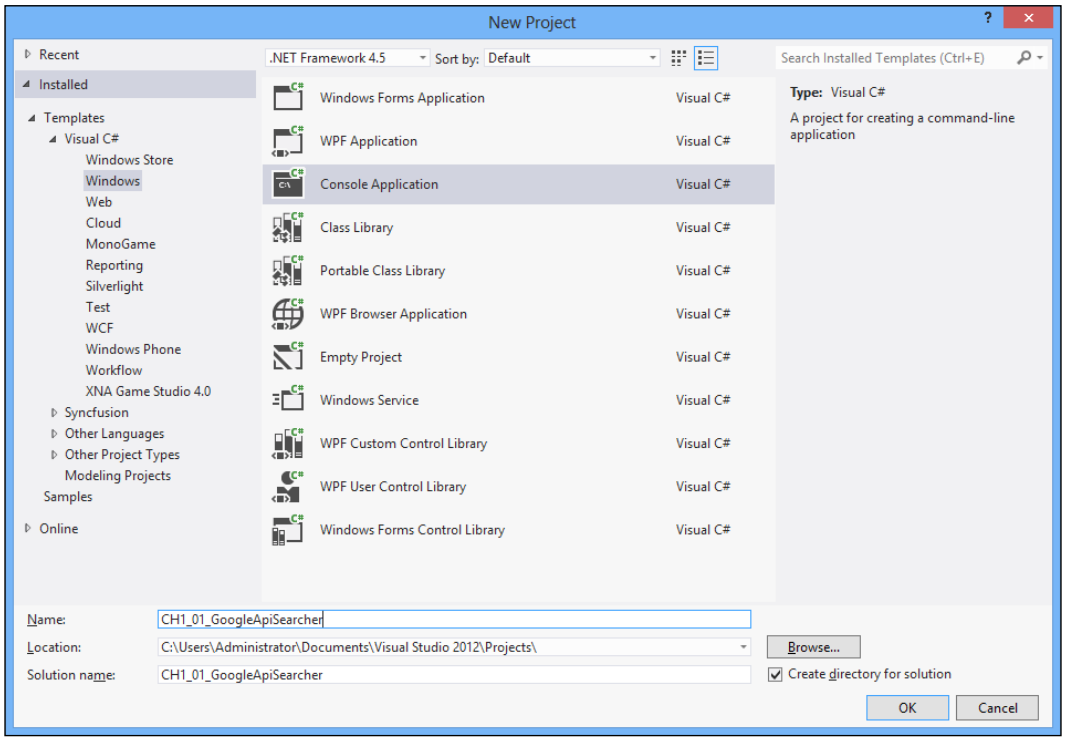
Manually adding third-party library references

In this chapter we will describe the most common way a developer finds a library and adds it to a solution. For the sake of simplicity, let us create a console application that will retrieve and parse an JSON result from a web service, for example, Google query search.

Let us start by creating a new console application:

1. In Visual Studio 2012, navigate to **File | New Project**.

2. In the **New Project** dialog, choose the **Console Application Visual C#** template.



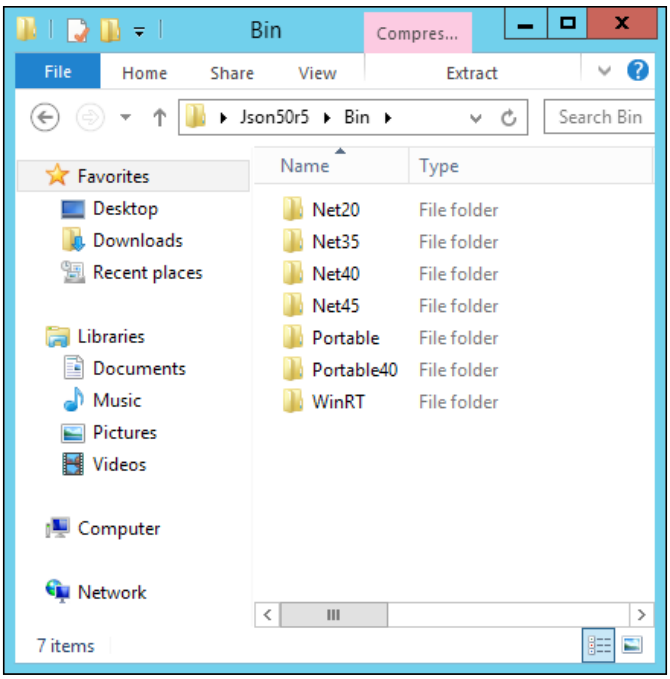
3. Let us name our solution `CH1_01_GoogleApiSearcher`, and start writing some code as shown:

```
class Program
{
    static void Main()
    {
        var request = "https://ajax.googleapis.com/
          ajax/services/search/web?v=1.0&q=nuget";
        using (var webClient = new WebClient())
        {
            var result =
              webClient.DownloadString(request);
            Console.WriteLine(result);
        }
    }
}
```

In the first line of the `Main` method, we store a HTTP request in a string, and it is used to query Google's Search API for search results for the `nuget` query. In the second line of this method, we are creating an instance of `WebClient`, making the web request, and then storing the result in a new string variable. The last line outputs the raw result to the console, in the form of a large unformatted JSON string.

Great, we have a JSON result. We now need a JSON library to parse our result into a meaningful form. When dealing with unknown libraries, the best choice is to type some relevant keywords into our favorite search engine.

For example, let's write `json for .net` into Google's search engine. The first hit on Google is **Json.NET - Home**, which sounds about right. As we all know, the first hit is not necessarily the best choice when searching for third-party libraries, but in this case we got lucky. Following the first hit, we are forwarded to `http://json.codeplex.com/`. After reading the description, we are sure that this is the appropriate library for our problem. So, we will find a download button and transfer the archived files to the computer. After opening the archived file, we will see two folders: `Bin` and `Source`. We decide that we will not be needing the library's source code; let us just choose the binary assembly and reference it from our project. In the `Bin` folder, we see another subset of folders with platform names as shown in the following screenshot; so again, we have to choose what to use in our code:



Since we are writing our application in .NET Framework 4.5, files in the `Net45` folder should be the right ones. But where should we put these files? There are no clear guidelines about how and where external references should be stored. In our root solution folder we should create a folder that will contain our external references. Let's name our newly created folder `ExternalReferences` and place the assemblies there.

Now we are set to add a reference in our project:

1. Choose **Add Reference** from the context menu in **Solution Explorer** on the **References** node.
2. Use the **Browse** button to select our assembly file from the file dialog box. The assembly file is located in the `ExternalReferences` folder in our project's root folder.
3. Click on the **OK** button to confirm the selection and add the assembly to the project.

Now we are ready to use the `Newtonsoft.Json` library for JSON manipulations. By changing our source code, we are able to get more useful information from our web service result. Let's make changes to our `Main` method, so it will look as shown:

```
static void Main(string[] args)
{
    var request = "https://ajax.googleapis.com/ajax/services/search/
web?v=1.0&q=nuget";
    string result;
    using (var webClient = new WebClient())
    {
        result = webClient.DownloadString(request);
    }
    // Deserializing JSON string to object
    dynamic data = JsonConvert.DeserializeObject(result);
    // Looping throughout results
    foreach (var item in data.responseData.results)
    {
       Console.WriteLine("{0} {1}", item.titleNoFormatting,
         item.url);
    }
}
```

Downloading the example code

You can download the example code files for all Packt books you have purchased from your account at `http://www.packtpub.com`. If you purchased this book elsewhere, you can visit `http://www.packtpub.com/support` and register to have the files e-mailed directly to you.

We used a method from our recently-added `JsonConvert.DeserializeObject` library. This code is written in .Net 4.5 and we can use the dynamic type for our result.

> The `dynamic` keyword is also available in .Net 4.0; more about the `dynamic` keyword is available at `http://bit.ly/csdynmc`.

In the `foreach` loop, we are looping through results and displaying the `titleNoFormatting` and `url` properties of results, which are our search results. Running the application outputs the following to the console window:

```
NuGet Gallery | Home http://nuget.org/

NuGet - Home http://nuget.codeplex.com/

NuGet - Wikipedia, the free encyclopedia
  http://en.wikipedia.org/wiki/NuGet

NuGet Package Manager extension - Visual Studio Gallery - Microsoft
  http://visualstudiogallery.msdn.microsoft.com/27077b70-9dad-4c64-
    adcf-c7cf6bc9970c
```

This library did not require editing of the configuration file, which is usually the most time-consuming step when using third-party libraries. Now, we have added a reference manually and successfully used it in our project. In the next chapter, we will learn how to set up NuGet in a Visual Studio environment; then we will take a look at how to add references using the NuGet Package Management.

Setting up NuGet in Visual Studio

This section will cover setting up NuGet in your development environment. If you are using Visual Studio 2012, you can skip it. NuGet client is a part of Visual Studio 2012 and it is fully integrated by default.

Requirements

Let's start with the operating system requirements. PowerShell 2.0 or later is required by NuGet Package Manager Console. It is already installed on the following operating systems:

- Windows 8/8.1
- Windows 7
- Windows Server 2008 R2

If you are using an older version of Windows, you have to install PowerShell 2.0 manually. This is required for the following systems:

- Windows XP SP3
- Windows Server 2008
- Windows Vista SP1
- Windows Server 2003 R2

> Installation of PowerShell 2.0 is not a subject of this book. The detailed PowerShell 2.0 installation procedure can be found at `http://bit.ly/PS20instll`.

Installing NuGet

Visual Studio 2010 requires installation of NuGet Package Manager. First let us check if the NuGet extension is already installed in your Visual Studio. If NuGet is already installed, there should be **Library Package Manager** under the **Tools** menu.

Installing from the Visual Studio gallery

You can install NuGet from Microsoft's Visual Studio gallery by performing the following steps:

1. Downloading and running the VSIX file from `http://bit.ly/ngtvsix` should bring up the VSIX Installer.
2. After selecting the Visual Studio instances where you wish to install NuGet Package Manager, you need to agree with the terms and click on the **Install** button.
3. After the installation is complete, a restart of Visual Studio is required. Once Visual Studio restarts, NuGet is ready to use.

Installing from the Extension Manager

We can install NuGet Package Manager from Visual Studio 2010 using the Extension Manager, as follows:

1. Navigating to **Tools | Extension Manager** brings up an **Extension Manager** dialog box.
2. Selecting the **Online Gallery** tab from the left side of the dialog box shows us all the available extensions from the Visual Studio Gallery.

www.allitebooks.com

3. Type `nuget` in the search box to find the **NuGet Package Manager** extension.
4. When **NuGet Package Manager** is selected, the **Download** button appears; and after clicking on it, we start the installation process described in the previous section.

Installing the command-line utility

By using the `NuGet.exe` command-line utility, we can interact with NuGet directly from the command line. The `NuGet.exe` utility is available for download at `http://bit.ly/nugetdl`. When the downloading is complete, we would recommend adding `NuGet.exe` to the `PATH` environment variable.

How to add a tool to the `PATH` environment is described at `http://bit.ly/pathset`.

Now it is time to open the command prompt and start using the NuGet command-line utility. Just by typing `nuget` we get a list of available commands.

Adding references to third-party libraries using NuGet

In this section, we will introduce a way of adding libraries with NuGet. Assuming you have successfully installed NuGet to your environment, you are now ready to start with a new project.

We are going to create a new empty console application, just like we did in the previous demo. In Visual Studio, perform the following steps:

1. Navigate to **File | New Project | Console Application**.
2. Name the project `CH1_02_GoogleApiSearcherNuget` and then confirm by clicking on the **OK** button.

When the project is created, a `Program.cs` source file with an empty `Main` method is opened in the text editor. In **Solution Explorer** our project's tree view is displayed. Right-clicking on the **References** node brings up the context menu, which contains three different menu items for adding a reference:

- **Add Reference…**
- **Add Service Reference**
- **Manage NuGet Packages…**

In the previous example we added a reference using the first option. Now our goal is to add a library using NuGet. Selecting **Manage NuGet Packages…** opens the dialog box, where we are managing all the packages related to the selected project. By default, the dialog opens on the **Installed packages** view, which is empty in our case since we have not added any packages yet. Selecting the **Online** option from the left-hand side menu lists every available package in the NuGet official package source. There are a lot of packages in this source; using the **Search online** textbox we can narrow down the results. Typing `json` into the textbox will automatically trigger a query on the NuGet feed, returning only the packages related to `json`. The list view in the middle of a dialog contains the search results. When selecting an item in the list view, the **Install** button appears. On the details pane on the right side, we can see all the important information about the selected package, for example, who is the creator, what is the package version, links to the project sites, library terms, and dependencies.

Let us select **Json.NET** and then click on the **Install** button. After a couple of progress bars, a green check icon appears on the package, meaning the package is installed in our project, as shown in the following screenshot:

Now we can re-use the code from the previous sample and compile the solution. Output is the same as that of the previous program, only this time we spent a lot less time for adding the reference.

By adding a NuGet package, we gained two files. The first file is the assembly under the `References` folder and the other is `packages.config` in our project's root. Information about our packages is stored in the `packages.config` file. For each package there is a unique package ID, version, and target framework version. We will see how NuGet uses this information in the next chapter.

Dependencies with NuGet

In this section, I would like to cover how NuGet handles dependencies. Let us continue with our solution from the previous section. If we want to create a portable class library from our project, we will not have the `WebClient` class available to us.

> Using the Portable Class Library project you can create assemblies that work on multiple .NET Framework platforms. More information about cross-platform development is available at `http://bit.ly/pcb_net`.

The problem is solved using the NuGet package called `Microsoft.Net.Http`. Perform the following steps:

1. In **Solution Explorer**, right-click on the **References** node and then select **Manage NuGet Packages…**.
2. Search for `httpclient` in the **Online** tab; when the search is complete, select **Microsoft HTTP Client Libraries**.

In the package summary window under **Description** and **Tags**, we can see that this package has two dependencies: **Microsoft.Bcl** and **Microsoft.Bcl.Build**. This means that these two libraries must be installed while installing the package **Microsoft HTTP Client Libraries**. Let us install this package and accept terms when prompted. This time we gained three extra lines in our `packages.config` file: one for our package and two dependency packages that are required for our package.

Now we can use `HttpClient` instead of `WebClient`. Instead of the following code:

```
using (var webClient = new WebClient())
{
   result = webClient.DownloadString(request);
}
```

Use the following code:

```
using (var webClient = new HttpClient())
{
    result = webClient.GetStringAsync(request).Result;
}
```

It should produce the same result. `GetStringAsync` is a task-based asynchronous pattern method. For the sake of simplicity, we are calling the `Result` property, which triggers synchronous execution. More about this pattern is available at `http://bit.ly/taskasync`.

Summary

This chapter has introduced NuGet and covered many of its benefits. We have briefly described the manageability problems with third-party libraries without NuGet and how NuGet solves these issues. We have also described the installation process for NuGet and all the tools required for the development environment. We added an external library manually by following a step-by-step tutorial. We concluded with a tutorial where we used NuGet to achieve the same result; we also used NuGet to add a library with dependencies on other libraries. In the next chapter, we will take a dive deep into the NuGet features such as updating referenced third-party libraries, multitarget platform support, and using NuGet with source control.

2
Package Management

In the previous chapter we have learned how to use NuGet to simplify adding references to third-party libraries from our projects. Now it's time to take a step further and see how else NuGet can make our lives easier during everyday development in Visual Studio. We will take a closer look at most of the supported features and show how they can be used to our advantage. At the end of the chapter, we will have a much deeper understanding of the working of the NuGet Package Manager and the problems it solves.

Updating referenced third-party libraries

It is painful to manually reference third-party libraries from our projects, as we have seen in the *Manually adding third-party library references* section of *Chapter 1*, *Using a Package Manager in Visual Studio*; the whole process would need to be repeated if we want to update the library to a newer version:

1. Visit the library download page and check the latest version available.
2. Navigate to the assembly referenced from our project and check its version.
3. Download the library again if its version is newer than the one we are currently referencing.
4. Extract the archive and replace the existing library files in our project with the new version.

Given the lengthiness of this process, we usually avoid it if we can; that is, we only update referenced third-party libraries if it is required. This is because there's a bug in the version we are using or we desperately need a newly available feature. By doing this, we deny ourselves the potential advantages of always using the latest stable version of the library such as:

- Fixes for issues that we haven't encountered or noticed yet
- Performance optimizations
- New noncritical features that could make things easier for us, and so on

Fortunately with NuGet, the process of updating referenced packages to the latest version is even simpler than adding the reference in the first place. There's no reason any more not to periodically check for package updates and decide whether to install them. Let's take a look at how we can do that.

Updating an installed package

To try updating the installed packages, we first need to prepare a project with an older version of the package installed by following these steps:

1. Create a new **Console Application** project in Visual Studio.
2. Open the **Package Manager Console** window by navigating to **Tools | Library Package Manager | Package Manager Console**.
3. Type the following command in the newly opened console window to install an old version of Json.NET:

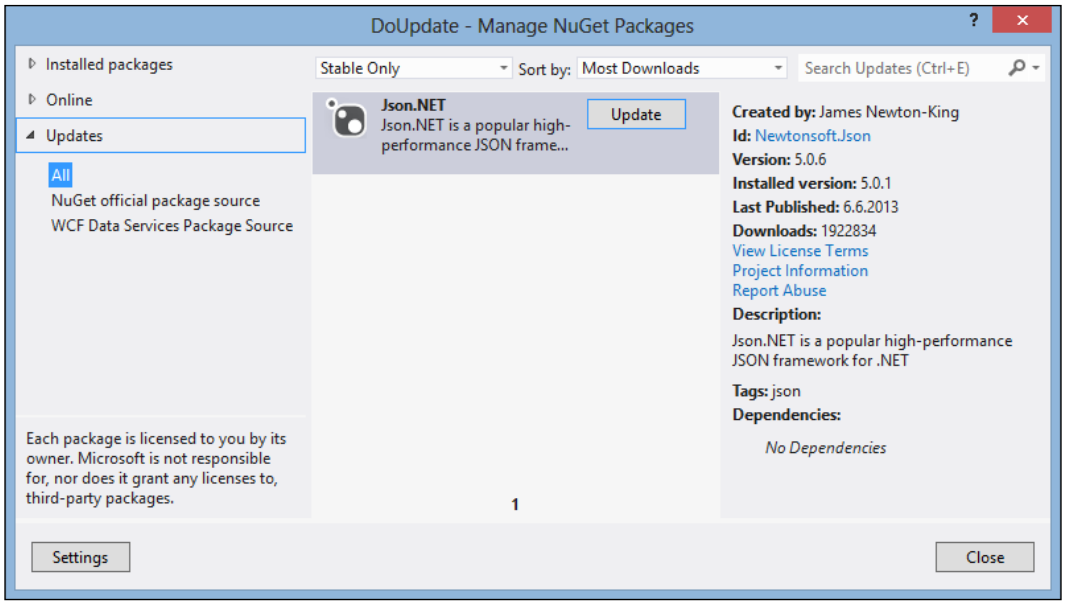
```
PM> Install-Package Newtonsoft.Json -Version 5.0.1
```

We have now created a solution referencing an older version of `Newtonsoft.Json` package. Usually, this will happen when a new version is released after we have already installed the package.

We have only used **Package Manager Console** to simulate the situation when we have an old version of a package installed in our solution. You usually won't need to do this during regular development. The usage of **Package Manager Console** and its commands will be explained in greater detail in the *Using Package Manager Console* section at the end of this chapter.

Let's try to update this package to its latest version:

1. Right-click on the project node in the **Solution Explorer** window and click on the **Manage NuGet Packages…** menu item.
2. In the left-hand side pane of the **Manage NuGet Packages** window, click on **Updates**. **Json.NET** should be the only package displayed in the list view. It will always contain all the packages that have a newer version available. In the right-hand side pane, information about the currently **Installed version** and the latest **Version** will be displayed for the currently selected package.

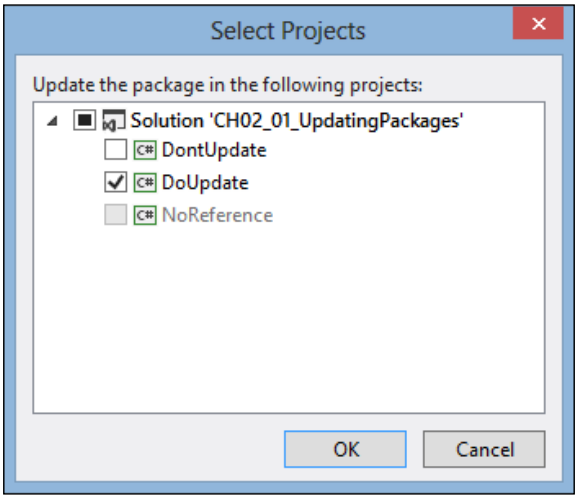


3. Click on the **Update** button in the central list view to update the installed package to the latest version.

> Notice the first drop-down above the central list view that has the **Stable Only** text displayed in it. If you click on it, you can change it to **Include Prerelease**. This will cause the search to include prerelease or beta versions of packages both when updating existing packages and when installing new ones.

In a solution with multiple projects, we don't need to repeat the preceding steps for each project. To perform the update for all projects at the same time, we right-click on the solution node in the **Solution Explorer** window and click on the **Manage NuGet Packages for Solution…** menu item.

4. A similar window will open with the only difference that there will be no information about the currently **Installed version** displayed. Clicking on the **Update** button will open a new window listing all of the projects in the solution:



Projects without the selected package installed will be displayed as disabled. For the remaining projects, we can select which ones we want to update by checking the checkboxes in front of them. By default, each one of them is selected for update, but sometimes we might not want to update the package for all of the projects in the solution at the same time; and this window allows us to do so.

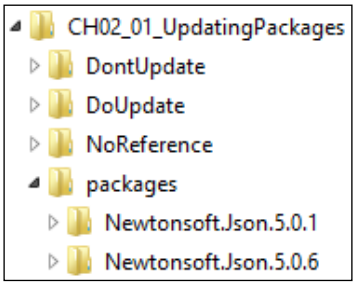
Local package version management

Before moving on, we'll take a closer look at how NuGet manages to keep a track of the package versions referenced by each project. When the first package is installed to a project, a `packages.config` file is added to it for this purpose. Let's take a look at its contents in the project we have just created, before we update the package to the latest version:

```
<?xml version="1.0" encoding="utf-8"?>
<packages>
  <package id="Newtonsoft.Json" version="5.0.1"
    targetFramework="net45" />
</packages>
```

For each installed package, a line similar to the highlighted one is added to the file. As you can see, it includes all the information about the package including the currently installed version. This allows NuGet to know exactly which version is installed for each project.

Of course, the referenced assemblies also need to be stored locally so that Visual Studio can reference them and use them during the build. By default, NuGet puts all of them inside the `packages` folder created in the solution folder as shown in the following screenshot. Sorting the packages by their unique ID and version allows NuGet to have any number of different versions in use at the same time.



Every time a package is installed or updated, NuGet modifies the references in that project. So, the references point to the assemblies inside the correct folder making sure the selected version will be used.

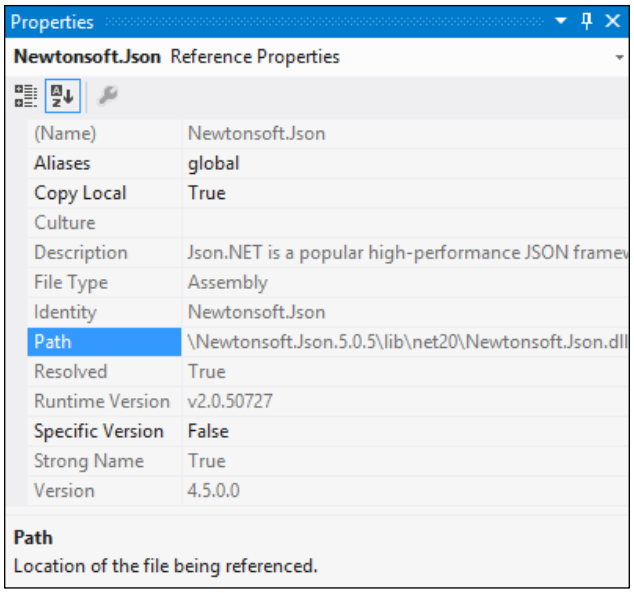
Support for different target platforms

As we have seen in the previous chapter, libraries can provide different assembly versions for different versions of the .NET Framework. NuGet takes care of that for us by automatically selecting the right assembly version.

Let's take a closer look at how this works:

1. Create a new **Console Application** project in Visual Studio. Make sure you select **.NET Framework 2.0** as the target platform in the **Add New Project** dialog box.
2. Right-click on the project node in the **Solution Explorer** window and click on the **Manage NuGet Packages…** menu item.
3. Search for **Json.NET** and click on the **Install** button for the **Json.NET** package.
4. Add a new **Console Application** project to the existing solution. This time, make sure you select **.NET Framework 4** as the target platform in the **Add New Project** dialog box.
5. Install the **Json.NET** package into this project as well by following steps 2 and 3 on its project node in the **Solution Explorer** window.

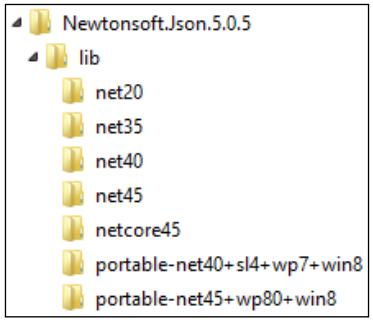
6. Now let's check which assembly got referenced in each project. To do that we'll open the **References** node for a project in the **Solution Explorer** window, click on the **Newtonsoft.Json** assembly, and press *Alt* + *Enter* to open the **Properties** window.



It turns out that the value of the `Path` property is different in each project:

- In the project targeting .NET Framework 2.0, it points to `packages\Newtonsoft.Json.5.0.5\lib\net20\Newtonsoft.Json.dll`
- In the project targeting .NET Framework 4, it points to `packages\Newtonsoft.Json.5.0.5\lib\net40\Newtonsoft.Json.dll`

We can see that in the package there is a separate folder for each supported target framework:



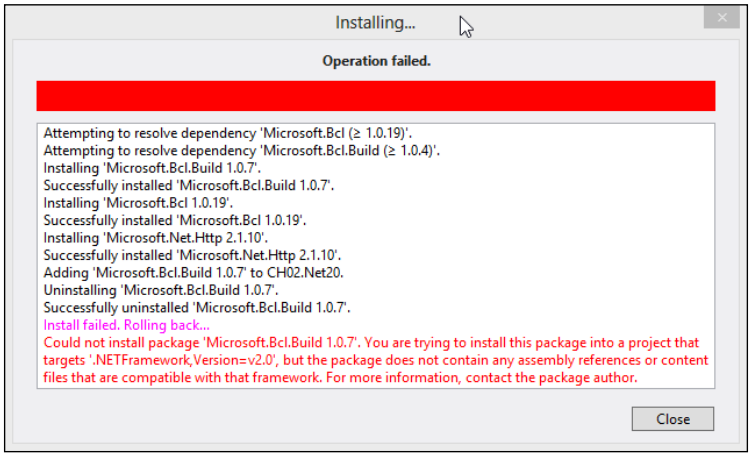
Installing a package for an unsupported target platform

What happens if we want to install a package into a project with a target platform that's not supported? Let's return to our console application targeting .NET Framework 2.0 and try installing Microsoft HTTP Client Libraries which are only available for newer versions of the .NET Framework:

1. Right-click on the project node in the **Solution Explorer** window and click on the **Manage NuGet Packages…** menu item.
2. Search for **Net.Http**. Since this is a part of the package ID, the first search result will be the package we are looking for, that is, **Microsoft HTTP Client Libraries**. As we can see, the library has come up in the list although it is not available for our target platform. NuGet search as of Version 2.5 doesn't take target platform into consideration and always returns the same results, no matter which platform our project targets.

> The package description on the right-hand side of the **Manage NuGet Packages** window lists the target platforms it supports, omitting .NET Framework 2.0, which should be an indication that the package won't work with our project. This is not enforced by NuGet, though; therefore, you will encounter many packages without any clear indication which target platforms they support.

3. Let's click on the **Install** button and see what happens. After accepting the license terms, the installation will start; but it fails soon, with an error message explaining that the package (or more precisely its dependent package `Microsoft.Bcl.Build`) isn't compatible with our target framework.



Although the search results aren't filtered based on the project target platform, NuGet is aware of it and prevents installing incompatible packages into a project, protecting us from issues that could arise because of it at a later time.

Using NuGet with source control

Source control systems are an integral part of software development. As soon as there is more than one person working on the project, it becomes an invaluable tool for sharing source code. Even when we are on the project on our own, there is no better way for tracking versions and source code changes. The question arises: how should we put the installed packages into source control?

The first impulse would be to simply add the `packages` folder to the repository. Though this will work, it isn't the best possible approach. Packages can grow quite large and they can be obtained from elsewhere; therefore, we would only "pollute" the repository with redundant files. Many source control systems don't handle large binary files well. Even for those that don't have such problems, having packages in the repository doesn't add much value; it does noticeably increase the repository size, though.

Fortunately, NuGet offers a feature called Package Restore, which can be used to avoid adding packages to source control. Let's see how it works.

> The following sample will use **Team Foundation Service** (**TFS**) for source control. If you don't have an account yet, you can sign up for free at `http://tfs.visualstudio.com`. You need a Microsoft account for authentication. If you decide to use a different source control system instead, just skip all the steps dealing with TFS, replacing them with equivalent actions in your source control system of choice.

We'll start by creating a sample project:

1. Create a new **Console Application** project in Visual Studio.
2. Install the Json.NET NuGet package into the project.
3. Add the following code to the `Main` method so that the project won't compile without a valid reference to the `Newtonsoft.Json.dll` assembly:

```
var jsonString =
    @"{
        ""title"": ""NuGet 2 Essentials"",
```

```
            ""authors"":""Damir Arh & Dejan Dakic"",
            ""publisher"": ""Packt Publishing""
        }";
var parsedJson = Newtonsoft.Json.JsonConvert
                            .DeserializeObject(jsonString);
```

4. Compile and run the project to make sure the code works.

It's time to create a source code repository. If you already have a repository, you can skip the following steps. Just make sure you have a repository ready and know how to connect to it before moving on.

You will need Visual Studio 2012 or Visual Studio 2010 with Service Pack 1 and KB2662296 installed to connect to TFS.

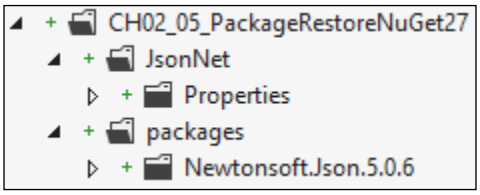
1. In a browser, navigate to `https://<accountname>.visualstudio.com/` (replacing `<accountname>` with the name you used when signing up for TFS).
2. Click on **New team project**.
3. In the **CREATE NEW TEAM PROJECT** dialog box, enter the project name (for example, `PackageRestore`) and click on **Create project**, leaving the rest of the fields unchanged.
4. Click on **Navigate to project** once the creation process is complete.
5. On the project page, click on **Open new instance of Visual Studio** in the **Activities** menu on the right to connect Visual Studio to your TFS account.
6. You can close this Visual Studio instance once the connection is established.

Now we're ready to add the project to the repository.

1. Return to Visual Studio, right-click on the solution node in the **Solution Explorer** window and click on the **Add Solution to Source Control…** menu item.
2. Make sure you select **Team Foundation Version Control** as the source control system if a dialog box pops up and asks you to make a selection.
3. In the **Connect to Team Foundation Server** dialog box which opens next, select your TFS account (for example, `accountname.visualstudio.com`) from the drop-down list and check the created repository in the **Team Projects** list box.
4. Click on **Connect** to move to the next step and confirm the default settings by clicking on **OK** in the dialog box that follows.

We still need to select the right set of files to add to source control and check them in so that they will be available for others.

1. Open the **Team Explorer** window and click on the **Source Control Explorer** link inside it.
2. Look in the tree view on the left side of the **Source Control Explorer** window. You will notice that apart from your project, the `packages` folder is also included. We need to exclude it since we don't want to add packages to source control.



3. Right-click on the `packages` folder and select **Undo Pending Changes…** from the context menu. Click on the **Undo Changes** button in the **Undo Pending Changes** dialog box that pops up.
4. Click on the **Check In** button in the toolbar and click on **Check In** again in the **Pending Changes** pane inside the **Team Explorer** window. Close the confirmation dialog box if it pops up. The `packages` folder should now be removed from the tree view.
5. Navigate to `https://<accountname>.visualstudio.com/DefaultCollection/<PackageRestore>/_versionControl` from your browser to check which files have been successfully checked in. Replace `<accountname>` and `<PackageRestore>` with appropriate values as necessary for your case.

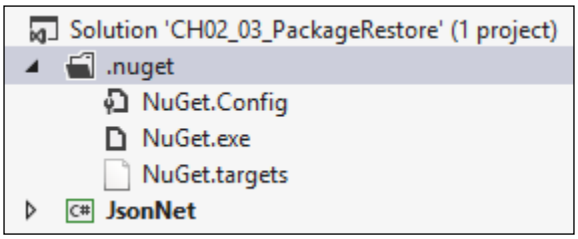
Let's retrieve the code and place it in a different location and see how the packages are going to get restored:

1. With TFS, a new workspace needs to be created for that purpose in the **Manage Workspaces** dialog box, which can be accessed by clicking on **Workspaces…** from the **Workspace** drop-down list in the **Source Control Explorer** toolbar.
2. Click on **Add…** to add a new workspace. You need to specify both **Source Control Folder** (solution folder, that is, `$/<PackageRestore>/<SolutionName>`) and **Local Folder** where you want to put the files.
3. After adding the workspace, confirm the next dialog box to get the files from the repository to your selected local folder.

4. Check the contents of that folder after Visual Studio is done to see that there is no `packages` folder inside it.
5. Open the solution from the new folder in Visual Studio and build it. You should notice a **NuGet Package Manager** dialog box popping up displaying the package restore progress and closing again once it is done. The application should build successfully. You can run it to see that it works as expected.
6. If you check the contents of the solution folder once again, you will see that the `packages` folder has been restored with all the required packages inside it.

Automatic Package Restore, which was described earlier has been available only since NuGet 2.7. In earlier versions, only **MSBuild-Integrated Package Restore** was supported. In case your repository will be accessed by users still using NuGet 2.6 or older, it might be a better idea to use this instead; otherwise, package restore won't work for them. You can enable it by following these steps (if you do this in NuGet 2.7, the Automatic Package Restore will get disabled):

1. Right-click on the solution node in the **Solution Explorer** window and click on the **Enable NuGet Package Restore** menu item.
2. Confirm the confirmation dialog box that pops up briefly explaining what is going to happen.
3. Another dialog box will pop up once the process is complete.
4. Notice that a `.nuget` folder containing three files has been added to the solution shown as follows:



When adding a solution configured like this to the source control, don't forget to include the `.nuget` folder as well. The `packages` folder of course still remains outside source control.

If you encounter a repository with MSBuild-Integrated Package Restore, which was enabled with NuGet 2.6 or older, the restoring of packages before build might fail with the following error:

```
Package restore is disabled by default. To give consent, open the Visual
Studio Options dialog, click on Package Manager node and check 'Allow
NuGet to download missing packages during build.' You can also give
consent by setting the environment variable 'EnableNuGetPackageRestore'
to 'true'.
```

This happens because the **Allow NuGet to download missing packages during build** setting was disabled by default in NuGet versions before 2.7. To fix the problem navigate to **Tools | Library | Package Manager | Package Manager Settings** to open the option dialog box on the right node; then uncheck and recheck the mentioned setting and click on **OK** to explicitly set it.

A more permanent solution is to either update `NuGet.exe` in the `.nuget` folder to the latest version or to switch to Automatic Package Restore instead as described at `http://bit.ly/NuGetAutoRestore`.

Using NuGet on a build server

Automatic Package Restore only works within Visual Studio. If you try to build such a solution on a build server by using MSBuild only, it will fail if the packages are missing. To solve this problem, you should use the **Command-Line Package Restore** approach by executing the following command as a separate step before building the solution file:

```
C:\> NuGet.exe restore path\to\SolutionFile.sln
```

This will restore all of the packages in the solution, making sure that the build won't fail because of missing packages. Even if the solution is using MSBuild-Integrated Package Restore, this approach will still work because all of the packages will already be available when it is invoked; and this will just silently be skipped.

The exact procedure for adding the extra step will depend on your existing build server configuration. You should either call it from within your build script or add it as an additional step to your build server configuration.

In any case, you need to make sure you have installed NuGet 2.7 on your build server as described in the *Installing the command-line utility* section of *Chapter 1, Using a Package Manager in Visual Studio.*

The NuGet Package Restore feature can be optimized even more on a build server by defining a common package repository for all solutions. This way each package will be downloaded only once even if it is used in multiple solutions; saving both the download time and the storage space.

To achieve this, save a `NuGet.config` file with the following content at the root folder containing all your solutions in its subfolders:

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <config>
    <add key="repositorypath" value="C:\path\to\repository" />
  </config>
</configuration>
```

You can even have more control of your repository location and other NuGet settings by taking advantage of the hierarchical or machine-wide `NuGet.config` file as explained at `http://bit.ly/NuGetConfig`.

Using Package Manager Console

We have already used Package Manager Console twice to achieve something that couldn't have been done using the graphical user interface. It's time to take a closer look at it and the commands that are available.

The **Package Manager Console** window is accessible by either navigating to **Tools | Library Package Manager | Package Manager Console** or by navigating to **View | Other Windows | Package Manager Console**. The most important commands are used to install, update, and uninstall packages on a project. By default, they operate on **Default project** selected from a drop-down list in the window's toolbar. The target project name can be specified using the `-ProjectName` parameter.

To get a list of all commands, type `Get-Help NuGet` in the console. To get more information about a command, type `Get-Help CommandName` in the console, replacing `CommandName` with the actual name of the command. You can also check the online PowerShell command reference at `http://bit.ly/NuGetPsRef`.

Let's take a look at few of the examples:

- To install the latest version of the `Newtonsoft.Json` package to the default project, type:

  ```
  PM> Install-Package Newtonsoft.Json
  ```

- To install Version 5.0.1 of the `Newtonsoft.Json` package to the default project, type:

  ```
  PM> Install-Package Newtonsoft.Json -Version 5.0.1
  ```

- To install the latest version of the `Newtonsoft.Json` package to the `Net40` project, type:

  ```
  PM> Install-Package Newtonsoft.Json -ProjectName Net40
  ```

- To update the `Newtonsoft.Json` package in all projects to its latest version, type:

  ```
  PM> Update-Package Newtonsoft.Json
  ```

- To update the `Newtonsoft.Json` package in all projects to Version 5.0.3 (this will fail for projects with a newer version already installed), type:

  ```
  PM> Update-Package Newtonsoft.Json -Version 5.0.3
  ```

- To update the `Newtonsoft.Json` package in the `Net40` project to the latest version, type:

  ```
  PM> Update-Package Newtonsoft.Json -ProjectName Net40
  ```

- To update all packages in all projects to the latest available version with the same major and minor version component, type:

  ```
  PM> Update-Package -Safe
  ```

- To uninstall the `Newtonsoft.Json` package from the default project, type:

  ```
  PM> Uninstall-Package Newtonsoft.Json
  ```

- To uninstall the `Newtonsoft.Json` package from the `Net40` project, type:

  ```
  PM> Uninstall-Package Newtonsoft.Json -ProjectName Net40
  ```

- To list all packages in the online package source matching the `Newtonsoft.Json` search filter, type:

  ```
  PM> Get-Package -ListAvailable -Filter Newtonsoft.Json
  ```

- To list all installed packages having an update in the online package source, type:

  ```
  PM> Get-Package -Updates
  ```

Installed packages can add their own commands. An example of such a package is `EntityFramework`. To get a list of all commands for a package, type `Get-Help PackageName`, replacing `PackageName` with the actual name of the package after it is installed, for example:

```
PM> Get-Help EntityFramework
```

Summary

This chapter has covered various NuGet features in detail. We started out with package versioning support and the package update process. We then moved on to built-in support for different target platforms. A large part of the chapter was dedicated to the usage of NuGet in conjunction with source control systems. We have seen how to avoid adding packages to source control and still have them automatically restored when they are required during build. We concluded the chapter with a quick overview of the console and the commands that give access to features not available using the graphical user interface.

This concludes our tour of NuGet from the package consumer point of view. In the following chapter, we will take on the role of a package creator and look at the basics of creating and publishing our own NuGet package.

www.allitebooks.com

3
Creating and Publishing a Package

In the previous chapter we took a closer look at NuGet's Package Management, and we learned how to use the advanced features provided by NuGet. The next step in our NuGet journey is actually creating our own packages. We will cover creating packages using a graphical interface and command line; we will also include package creation during the build process. Later on, we will discuss package versioning and publishing. At the end of this chapter, you will have all the knowledge necessary to create and publish packages in the official NuGet source.

As mentioned before, we can create a NuGet package using a GUI tool or command line. First, we will cover package creation through the interactive GUI and then we will do the same with the Command Prompt commands. A non-interactive creation process is the recommended choice for integration in any build automation process, be it Visual Studio or a continuous integration system.

Creating a package with NuGet Package Explorer

With NuGet Package Explorer we can create and publish NuGet packages with ease. This application allows us also to analyze and verify packages. It also provides helpful tips for creating packages. The application is available for download at `http://bit.ly/ngtpexp`. This is a **ClickOnce** application, which means no installation steps are required and it keeps the application updated.

More about creating and deploying ClickOnce applications is available at `http://bit.ly/clickonceapp`.

We need to create an assembly that we will pack in our NuGet package. To simplify this process, we will use and modify the assembly that we created in *Chapter 1*, *Using a Package Manager in Visual Studio*. First, let us change the application's output type to class library:

1. Right-click on the project node in the **Solution Explorer** window and click on **Properties**.
2. In the left pane, select **Application**.
3. In the **Output type** drop-down list, change the value from **Console application** to **Class Library**.
4. Press *Ctrl* + *S* to save the changes.

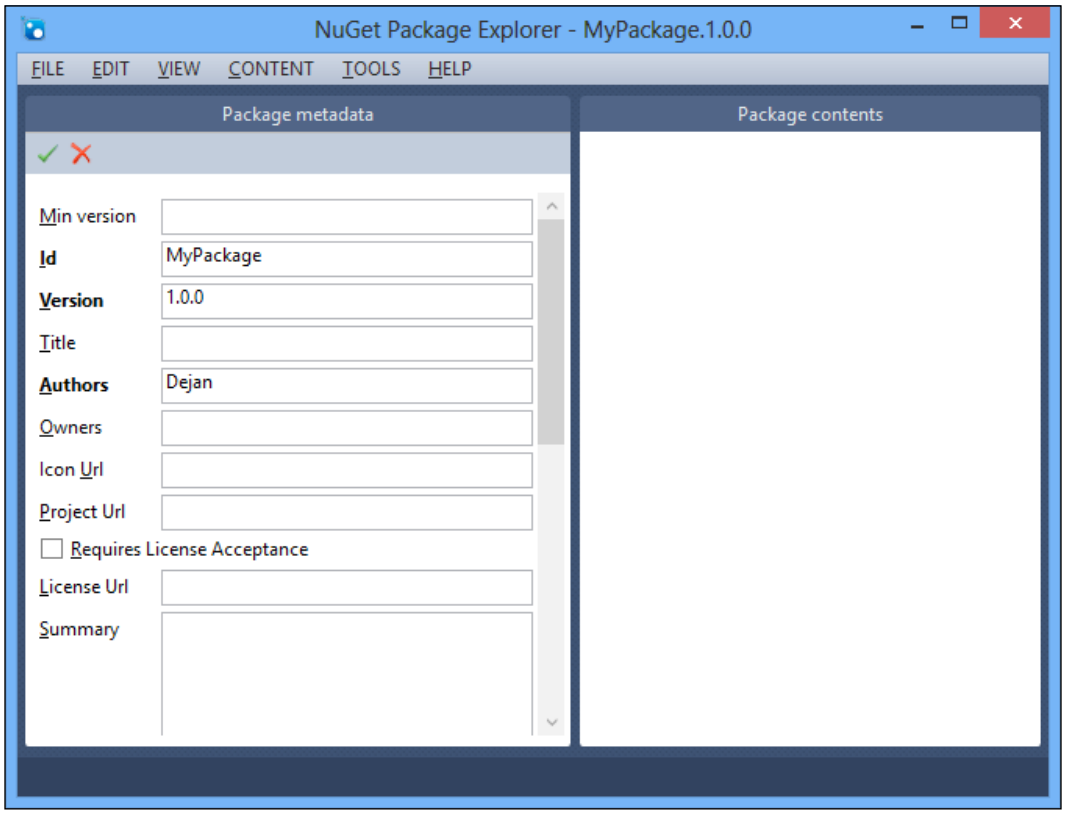
Now we have to modify the code so that it will behave like an actual library. We need to change the class and methods names, method signature, and body.

```
public static class CH3_01_ClassLibrary
{
    public static List<string> Query(string searchQuery)
    {
        var request = String.Format
            ("https://ajax.googleapis.com/
              ajax/services/search/web?v=1.0&q={0}",
             Uri.EscapeUriString(searchQuery));
        string result;
        using (var webClient = new WebClient())
        {
            result = webClient.DownloadString(request);
        }

        // Deserializing JSON string to object
        dynamic data = JsonConvert.DeserializeObject(result);

        var returnList = new List<string>();
        // Looping through the results
        foreach (var item in data.responseData.results)
        {
            returnList.Add(String.Format("{0} {1}",
              item.titleNoFormatting, item.url));
        }
        return returnList;
    }
}
```

This is how our class should look like, we are using only the `Newtonsoft.Json` library. Let's build it first by performing the following steps:

1. Open the **Configuration Manager** window by navigating to **Build | Configuration Manager…**.
2. Set the release configuration in the **Configuration Manager** window.
3. Compile the solution by navigating to **Build | Build Solution**.

Now, it is time to start packaging our solution. Run the **NuGet Package Explorer** and select **Create a new package** from **Common tasks**, or press *Ctrl* + *N*. An empty metadata file is displayed; navigating to **EDIT | Edit Metadata** or pressing *Ctrl* + *K* toggles the editing mode. An empty manifest file should appear on the screen as shown in the following screenshot:



A NuGet specification file is in the XML format. The information about the specification's metadata is described in the following table.

XML element	Description
`id`	This element is required. It is the unique identifier of the package. This is the package name that is shown in the **Package Manager Console** or **Manage NuGet Packages** dialog boxes. Rules for naming IDs are the same as rules for .NET namespaces.
`version`	This element is required. It is the package version, for example, `1.2.3.4`. More about versioning will be covered in the following section.
`title`	It is a friendly package name. If no title is specified, the ID element is used instead.
`authors`	This element is required. This shows package authors listed in a comma-separated format.
`owners`	This shows package owners listed in a comma-separated format. This element is ignored when uploading to the NuGet Gallery.
`iconUrl`	This is the online URL for the package's icon. The icon should have a transparent background, 32 x 32 pixel size, and should be stored in the PNG file format.
`projectUrl`	This is a URL to the package's home page.
`licenseUrl`	This is a URL to the package's license and agreement terms. The following URL can help you decide which license to choose: `http://choosealicense.com/`.
`requireLicence Acceptance`	This specifies whether the license (from the `licenceUrl` element) must be accepted before the package is installed. This is a Boolean value.
`summary`	This is a short description of a package. If specified, this message is displayed in the middle pane of the **Manage NuGet Packages** dialog box. Otherwise, the truncated version of the long description is used instead.
`description`	This element is required. This is a long description of the package. This message is displayed in the right pane of the **Manage NuGet Packages** dialog box and also in **Package Manager Console** when using the `Get-Package` command.
`copyright`	Copyright information for the package.
`tags`	Technical keywords and tags that describe the package in a space-delimited format. This information is used for searching in the **Manage NuGet Packages** dialog box or from **Package Manager Console** using the `Get-Package –ListAvailable` command.

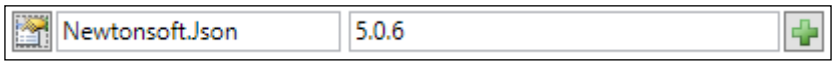
XML element	Description
`language`	This is the package's locale ID, for example, `en-us`.
`dependencies`	This is the dependency list for the package. More about this element will be discussed in the next section.
`framework Assemblies`	These are the .NET Framework assemblies that are required for the package. If a package depends on a .NET Framework assembly, this element allows you to specify set Framework assemblies in the GAC. Note that these assemblies are not included in the package as they are part of the .NET Framework.
`minClientVersion`	This is the minimum version of the NuGet client that can install this package. This is available in NuGet Version 2.5 and above.

The official NuSpec reference is available at `http://bit.ly/nugetSpec`.

Let's create a package by entering the necessary information:

1. Set the **Id** value to `MyDemoPackage`.
2. Set the **Version** value `1.0.0.0`.
3. Set **Authors** to your name.
4. Set **Description** to `My first package`.
5. Click on the **Edit dependencies** button and the **Package Dependency Editor** dialog box shows up.
    1. Click on the green plus icon to add a new empty framework group `net40`. More about the framework groups will be covered in the next chapter.
    2. Click on the **Select dependency from NuGet feed** button and type `json` in the search field.
    3. Select `Newtonsoft.Json` and click on **OK**.

    The selected package's ID and version values appear in textboxes. Confirm that you are adding the dependency by clicking on the plus sign as shown in the following screenshot:



6. By navigating to **CONTENT | Add | Lib folder**, add a library folder that will contain assemblies.
7. Since we are creating a package for .NET Framework Version 4.0, we will create a folder for this specific .NET version. Right-click on the library folder in the **Package contents** pane and navigate to **Add .NET folder | v4.0**.

8. Add the assembly to the library folder by selecting the `net40` folder in the **Package contents** pane and then navigate to **CONTENT | Add | Add existing file…** and select the assembly file generated from the preceding build.
9. To analyze the package, navigate to **TOOLS | Analyze package** or press *Ctrl* + *Q*. The **Package Analysis** pane should appear in **NuGet Package Explorer** with the message, **0 issue(s) found**.
10. Press *Ctrl* + *S* to save the package.
11. Now, we have created a simple package. Let us check the generated manifest file before publishing the package. Navigate to **EDIT | Edit Metadata Source** and we should see an XML (such as name elements) similar to the following:

```
<?xml version="1.0" encoding="utf-8"?>
<package xmlns="http://schemas.microsoft.com/packaging/2010/07/
nuspec.xsd">
    <metadata>
        <id>MyDemoPackage</id>
        <version>1.0.0</version>
        <title>NuGet 2 Essentials Demo Package</title>
        <authors>Damir Arh, Dejan Dakic</authors>
        <requireLicenseAcceptance>
            false
        </requireLicenseAcceptance>
        <description>
            Example from NuGet 2 Essentials
        </description>
        <dependencies>
            <dependency id="Newtonsoft.Json"
              version="5.0.6" />
        </dependencies>
    </metadata>
</package>
```

Dependencies

The dependencies element in the NuSpec file contains a set of dependency elements. Each element is a reference to another NuGet package. In line 11 of our specification XML, we can see that our NuGet package depends on the `Newtonsoft.Json` package, Version 5.0.6.

When our package is installed, NuGet will automatically resolve and install all specified dependencies (`Newtonsoft.Json` in our case).

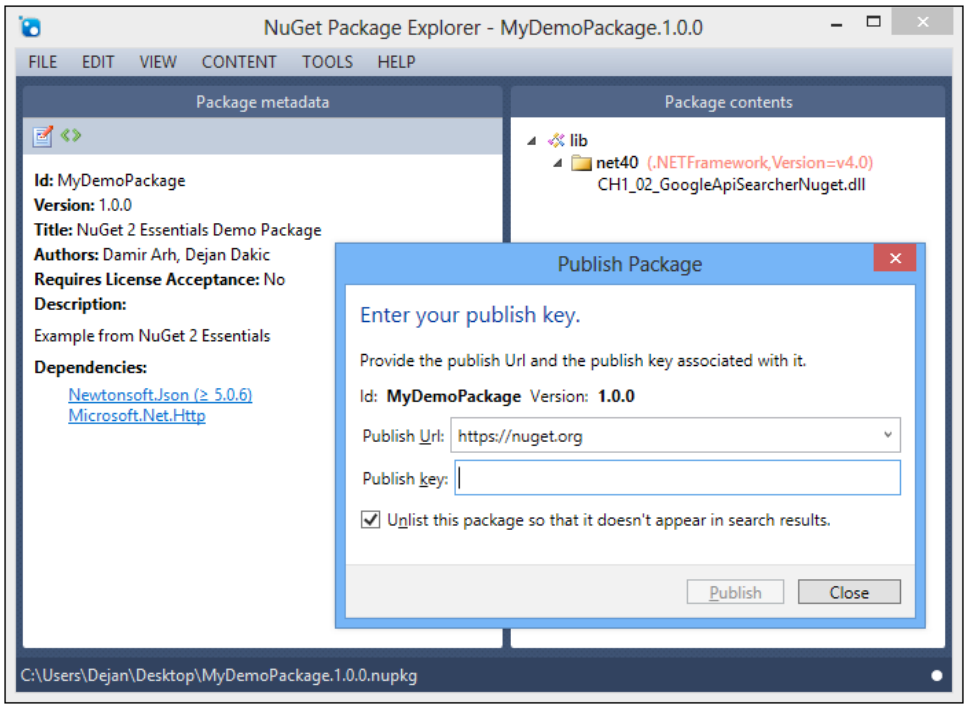
The version attribute represents a minimum version of a required package; if the attribute is empty or left out, the latest version is used. You can specify a more explicit version range.

Extended information about specifying dependencies can be found at `http://bit.ly/nugetDeps`.

Publishing the package to the official NuGet Gallery

It is time that we publish the package on the NuGet Gallery. This sample will use the NuGet Gallery.

1. If you do not have an account yet, you can sign up for free at `https://www.nuget.org/users/account/LogOn`. When our username on the NuGet Gallery is activated, we can access its API key. The API key is a secret key that provides us with the token that identifies us in the gallery. If the token is accidentally revealed, you can regenerate your token at any time.
2. In **NuGet Package Explorer**, load the previously generated package and navigate to **FILE | Publish**, or press *CTRL + P*.



3. Enter the API key in the **Publish key:** field and click on **Publish**.
4. Check **Unlist this package so that it doesn't appear in the search results.** to keep the package private and unlisted. Since `MyDemoPackage` is already taken, the publishing will fail with the following error:

```
Unable to process request. 'A package with id 'MyDemoPackage'
and version '1.0.0' already exist and cannot be modified.'
The remote server returned an error: (409) Conflict.
```

Let us change the package ID and try again. If we choose an available package ID, our package gets published successfully.

Managing packages on the NuGet Gallery can be done at `https://nuget.org/account/Packages`, where our published package should also be listed. There we can control how packages are listed and check the package statistics.

Creating and publishing a package from the command line

In this section, we will repeat the steps of creating and publishing the package from previous sections; only this time we will create and publish from the command line.

Creating a NuGet specification

Let us create an empty directory and then type the following command in the newly-created `nuget spec MyDemoPackageCmd` directory.

```
D:\MyDemoPackageCmd\a>nuget spec MyDemoPackageCmd
Created 'MyDemoPackageCmd.nuspec' successfully.
```

Now let us open this file with a text or an XML editor. We can see the specification file in the XML format; we had generated this file by editing the necessary fields using **NuGet Package Explorer**. This `nuspec` file contains all the elements with default values. A few of the values are automatically generated for us, for example, ID and name element. Let us just copy the generated XML from **NuGet Package Explorer**, but do not forget to change the ID to `MyDemoPackageCmd`. We could customize the specification file in depth, if needed.

A `nuspec` file can be generated directly from the assembly that is going to be packed, which has the advantage that more parameters could be automatically filled into the specification. This can be achieved with the `nuget spec -a assembly.dll` command.

Packing the package

With our updated `nuspec` file, we are ready to add files to the package. We have to create a library folder and a framework version folder for the assembly. We can do this by typing `mkdir lib\net40`.

1. Copy the assembly file to this directory. Typing `dir /B /S` should produce a result similar to what is shown here:

```
D:\MyDemoPackageCmd>dir /B /S
D:\MyDemoPackageCmd\lib
D:\MyDemoPackageCmd\MyDemoPackageCmd.nuspec
D:\MyDemoPackageCmd\lib\net40
D:\MyDemoPackageCmd\lib\net40\CH1_02_GoogleApiSearcherNuget.dll
D:\MyDemoPackageCmd>
```

2. Now we can pack our package by using the `nuget pack MyDemoPackageCmd.nuspec` command. The command-line tool gives the following output:

```
Attempting to build package from 'MyDemoPackageCmd.nuspec'.
Successfully created package
'D:\MyDemoPackageCmd\MyDemoPackageCmd.1.0.0.nupkg'.
```

3. We gained a package ready for publishing in the NuGet Gallery.

We could also create a package by referencing the project configuration file. Let us change the directory to our project folder. Type `nuget pack CH1_02_GoogleApiSearcherNuget.csproj` and the NuGet command-line utility will generate a package with the required specification.

Publishing the package

Before we publish the package, we must set the API Key in our NuGet command-line utility by using the `nuget setapikey <your private API key>` command.

Now we are ready to publish the package from the command line by typing `nuget push MyDemoPackageCmd.1.0.0.nupkg`.

```
D:\MyDemoPackageCmd>nuget push MyDemoPackageCmd.1.0.0.nupkg
Pushing MyDemoPackageCmd 1.0.0 to the NuGet gallery
(https://www.nuget.org)...
Your package was pushed.
```

Now, we have successfully published the package using only the Command Prompt commands.

Generating the package during the build process

In this section we will automatically create the NuGet package upon every build. We will use the Visual Studio post-build event to generate the NuGet package every time the solution is built.

Before we start using the post-build event, we have to make slight changes to our solution. In *Chapter 2*, *Package Management*, we covered how to enable the NuGet package restore functionality. With this functionality we will gain access to the `nuget.exe` command-line utility tool that will be used for packing our assembly during the build process. Let's start with the necessary preparations:

1. In the **Solution Explorer** window, right-click on the solution, select **Enable NuGet Package Restore**, and confirm the action.
2. We need to modify the `AssemblyInfo.cs` file and edit the assembly attributes. These attributes will map with the NuGet package.
3. Open the `AssemblyInfo.cs` file by expanding the project's **Properties** node in the **Solution Explorer** window.

The following table describes the mapping relation between assembly attributes and the NuGet specification elements:

.NET Framework Attribute	NuGet specification XML element
AssemblyTitleAttribute	`id`
AssemblyDescriptionAttribute	`description`
AssemblyCompanyAttribute	`authors`, `owners`
AssemblyCopyrightAttribute	`copyright`
AssemblyVersionAttribute	`version`

Our `AssemblyInfo.cs` file should look something like as shown:

```
using System.Reflection;
using System.Runtime.InteropServices;

[assembly:
  AssemblyTitle("CH3_02_AutomatedPackageDuringBuildProcess")]
[assembly: AssemblyDescription("This is a description")]
[assembly: AssemblyConfiguration("")]
[assembly: AssemblyCompany("Your name")]
[assembly: AssemblyProduct("")]
[assembly: AssemblyCopyright("Copyright © 2013")]
```

```
[assembly: AssemblyTrademark("")]
[assembly: AssemblyCulture("en-us")]
[assembly: Guid("c81c979d-853e-46ef-910c-09909a571d5e")]
[assembly: AssemblyVersion("1.0.0.*")]
```

Note the last line, `[assembly: AssemblyVersion("1.0.0.*")]`. By using an asterisk `*` for the revision number, every change in the assembly will create a unique revision number. Recompile the solution and check the `bin\Debug` folder for the assembly file version. Our assembly file version will be `1.0.0.XXXXX`, as shown in the following screenshot. We will cover more about versioning in the following section, *Understanding package versioning*.

Name	Type	File version
CH3_02_AutomatedPackageDuringBuildProcess.dll	Application extens...	1.0.0.40051
CH3_02_AutomatedPackageDuringBuildProcess.dll.config	CONFIG File	
CH3_02_AutomatedPackageDuringBuildProcess.pdb	Program Debug D...	
Newtonsoft.Json.dll	Application extens...	5.0.6.16206
Newtonsoft.Json.xml	XML File	

Now we can add a post-build event to generate the package. We should create an output directory for automatically generated packages. Let us create `AutoGeneratedPackages` in our solution root directory.

1. In the **Solution Explorer** window, right-click on the project and select **Properties**. In the left-hand pane, select the **Build event** tab and in **Post-Build event command line** write the following code:

   ```
   "$(SolutionDir)\.nuget\NuGet.exe" pack "$(SolutionDir)\
   CH3_02_AutomatedPackageDuringBuildProcess\CH3_02_
   AutomatedPackageDuringBuildProcess.csproj" -OutputDirectory
   "$(SolutionDir)\AutoGeneratedPackages"
   ```

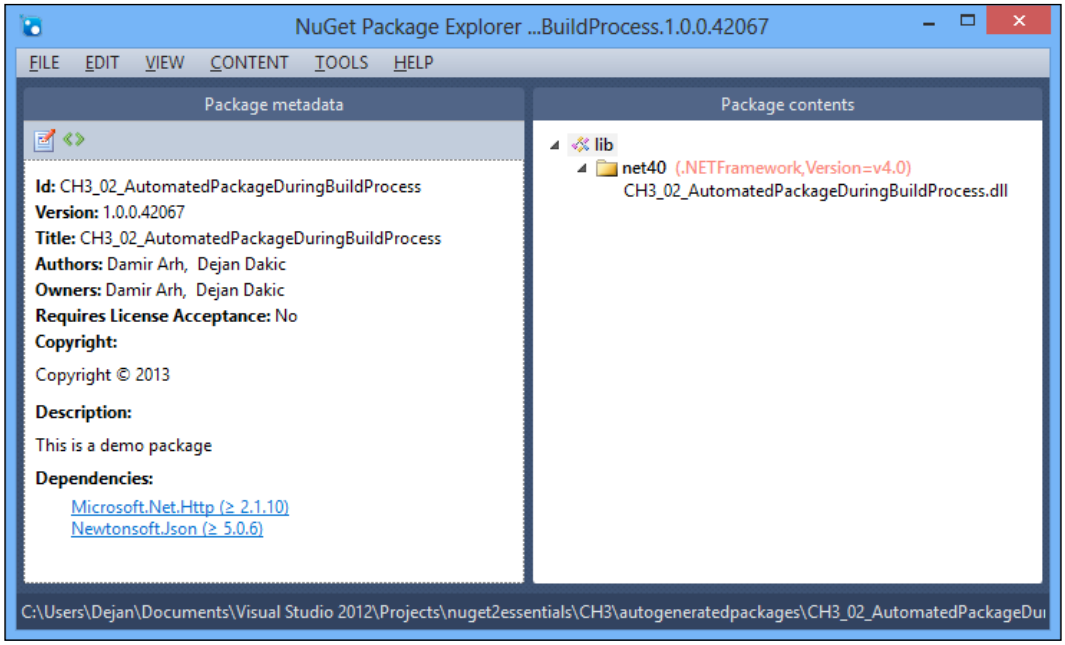
   [This code should be written in a single line without any new lines.]

2. Save the changes with *Ctrl* + *S* and close the **Properties** tab. Now when you build the project again, build output should be similar to:

   ```
   1>------ Build started: Project: CH3_02_
   AutomatedPackageDuringBuildProcess, Configuration: Debug Any CPU
   ------
   1>  All packages listed in packages.config are already installed.
   ```

```
1>  CH3_02_AutomatedPackageDuringBuildProcess -> C:\Users\
Dejan\Documents\Visual Studio 2012\Projects\nuget2essentials\
CH3\CH3_02_AutomatedPackageDuringBuildProcess\bin\Debug\CH3_02_
AutomatedPackageDuringBuildProcess.dll

1>  Attempting to build package from 'CH3_02_
AutomatedPackageDuringBuildProcess.csproj'.

1>  Packing files from 'C:\Users\Dejan\Documents\
Visual Studio 2012\Projects\nuget2essentials\CH3\CH3_02_
AutomatedPackageDuringBuildProcess\bin\Debug'.

1>  Found packages.config. Using packages listed as dependencies

1>  Successfully created package 'C:\Users\Dejan\Documents\Visual
Studio 2012\Projects\nuget2essentials\CH3\\AutoGeneratedPackages\
CH3_02_AutomatedPackageDuringBuildProcess.1.0.0.41851.nupkg'.

========== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped
==========
```

3. In the `AutoGeneratedPackages` directory, there will be the NuGet package we created. Open this package with **NuGet Package Explorer** to examine it.



Understanding package versioning

Every NuGet package has its version, in this section we will cover how to version our package. As of NuGet 1.6, NuGet supports versioning according to the **Semantic Versioning** (**SemVer**) specification.

SemVer

SemVer is a set of rules for versioning your APIs. Every version number has a very specific meaning. Each version number consists of three parts in the dot-separated format, **MAJOR.MINOR.PATCH**.

Very brief rules for changing a version:

- A major number should be incremented when incompatible changes are made to the API
- A minor number should be incremented when you add functionality or features and keep the API as backward compatible
- A patch number should be incremented when you want to make bug fixes and also keep the API as backward compatible

Semantic Versioning is described in depth at `http://semver.org`.

The prerelease packages

The default NuGet behavior provides stable packages; but as covered in the *Updating referenced third-party libraries* section of *Chapter 2*, *Package Management*, NuGet also provides functionality for prerelease packages.

The prerelease versions are denoted by appending an additional string to the patch number separated by a dash. The suffix content is not important; if there is a dash separator, it is a prerelease version. The following are examples of prelease versions:

- 1.0.0-alpha
- 1.0.0-beta
- 1.0.0-rc

Precedence of the prerelease versions is determined in alphabetical order. Stable Version 1.0.0 is larger than 1.0.0-beta.

The following is an example of the precedence order from the highest to the lowest versions:

- 1.0.0
- 1.0.0-zombie
- 1.0.0-rc
- 1.0.0-beta3
- 1.0.0-beta

Creating a prerelease NuGet package is nothing more than adding a prerelease string to a version. This can be achieved in two ways:

- In the NuSpec file, specify the version in the `<version />` element. For example, `<version>1.0.0-rc</version>`.
- Using `AssemblyInformationalVersionAttribute` in the `AssemblyInfo.cs` file. For Example, `[assembly: AssemblyInformationalVersion("1.0.0-rc")]`.

Summary

This chapter has covered the creation of NuGet packages. We started by modifying our sample application to the class library that looks like an API. We continued with the graphical tool NuGet Package Explorer, where we analyzed the NuGet specification file. Using NuGet Package Explorer, we created the NuGet package. We have also seen how to obtain the API Key from the official NuGet Gallery where we have published our NuGet package. Then, we repeated the whole package generation process with the `nuget.exe` command-line utility tool—NuGet. We have also seen how to use Visual Studio post-build events to automatically generate package. We concluded the chapter with a brief introduction to package versioning.

4

Adding Features to a NuGet Package

Now that we have created our first NuGet package, we are ready to learn about the other features supported by the NuGet Package Manager. We have already encountered some of them while consuming packages in *Chapter 2*, *Package Management*. Now, it's time to take advantage of these features in our own packages as well as of several others that we haven't mentioned before. By the end of this chapter, we will have a complete overview of the different types of files that are usually included in the package and what they mean with respect to the behavior of the package.

Package folder structure

Let's start by inspecting the contents of a NuGet package. Although the file has a `nupkg` extension, it's actually just a ZIP archive that contains files organized in different folders following a predefined convention. If you change the package's file extension to `.zip` and unpack it, you will find a subset of the following folders inside it:

- The `lib` folder that contains the assemblies that need to be referenced from the project; this is where our assemblies ended up in the package created in the previous chapter
- The `content` folder that contains the files that need to be added to the project; usually these are source files or resources such as images, and other files
- The `build` folder that contains the MSBuild files that need to be included in the project file; they define additional targets and properties to be used during the build process
- The `tools` folder that contains files not directly used in the project; typically, these are PowerShell scripts and executable files that can be called from the NuGet Package Manager Console

Also, at the root of the package, there's always a `nuspec` manifest file containing the package metadata as described in the previous chapter.

> A NuGet package contains other files and folders that are not of interest to us at this point. The `_rels` and `package` folders and the `[Content_Types.xml]` file are automatically generated when the package is built; other files with no built-in meaning for NuGet, such as license texts, can also be added to the root of the package.

To take advantage of everything that the folders offer, the same folder structure with all the files inside them first needs to be created in the local filesystem along with the corresponding `nuspec` manifest file. A package can then be created by calling `NuGet.exe` with the following parameters:

```
C:\> NuGet.exe pack MyAssembly.nuspec -BasePath path\to\created\folder\
structure -OutputDirectory path\to\package\output\directory
```

Of course, you will have to replace `MyAssembly.nuspec, path\to\created\folder\structure` and `path\to\package\output\directory` with appropriate values for your case.

The rest of this chapter provides a more detailed look at each of the folders listed earlier and everything that they support.

Multiplatform support

In the previous chapter, we have already put the assembly in the `net40` subfolder of the `lib` folder because it was designed for the .NET Framework 4.0. This automatically meant that our package could only be installed into projects targeting this or a newer framework version. As shown in *Chapter 2*, *Package Management*, NuGet packages can include multiple different versions of assemblies for different versions of the .NET Framework. We'll take a look at how to create such a package.

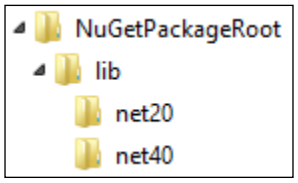
Let's start with the sample library from the previous chapter. We'll keep its current implementation that works only on the .NET Framework 4.0 and higher unchanged, but we'll create another version that will work with older versions of the .NET Framework as well. Have a look at the following code:

```
public class GoogleSearchApi
{
    public static List<string> Query(string searchQuery)
    {
        string request = String.Format
          ("https://ajax.googleapis.com/ajax/services/
            search/web?v=1.0&q={0}",
```

```
            Uri.EscapeUriString(searchQuery));
        string result;
        using (WebClient webClient = new WebClient())
        {
            result = webClient.DownloadString(request);
        }

        // Deserializing JSON string to object
        JObject data = JObject.Parse(result);

        List<string> returnList = new List<string>();
        // Looping through the results
        foreach (JObject item in data["responseData"]["results"])
        {
            returnList.Add(String.Format("{0} {1}",
                item["titleNoFormatting"].Value<string>(),
                item["url"].Value<string>()));
        }
        return returnList;
    }
}
```

The changes are highlighted in the preceding code. The unsupported `dynamic` features were replaced with a statically typed approach using an indexer based on `string`. This code gets compiled successfully under .NET Framework 2.0 and can be used with any projects targeting it.

Now that we have created two different assemblies with the same functionality for two different target platforms, we need to organize them into a correct folder structure. As you can expect, both assemblies will end up inside the `lib` folder (not directly), but each one will be inside a subfolder representing its target platform: `net20` for the .NET Framework 2.0 assembly and `net40` for the .NET Framework 4.0 assembly. The following screenshot shows the required folder structure:



We don't need to make any changes to the `nuspec` file, except for the package ID value that must match the filename `GoogleSearchApi` without its extension. We also needn't put the file in the `NuGetPackageRoot` folder. It will be put there by `NuGet.exe` when we build the package.

Once we have prepared the folder structure, we can create the package by calling `NuGet.exe`. Assuming there is a `GoogleSearchApi.nuspec` file in the solution folder and that the `NuGetPackageRoot` folder is in the solution folder as well, the following command line needs to be executed from the solution folder to create the `GoogleSearchApi.1.0.0.nupkg` package in the same folder:

```
C:\SolutionDir> NuGet.exe pack GoogleSearchApi.nuspec -BasePath
NuGetPackageRoot
```

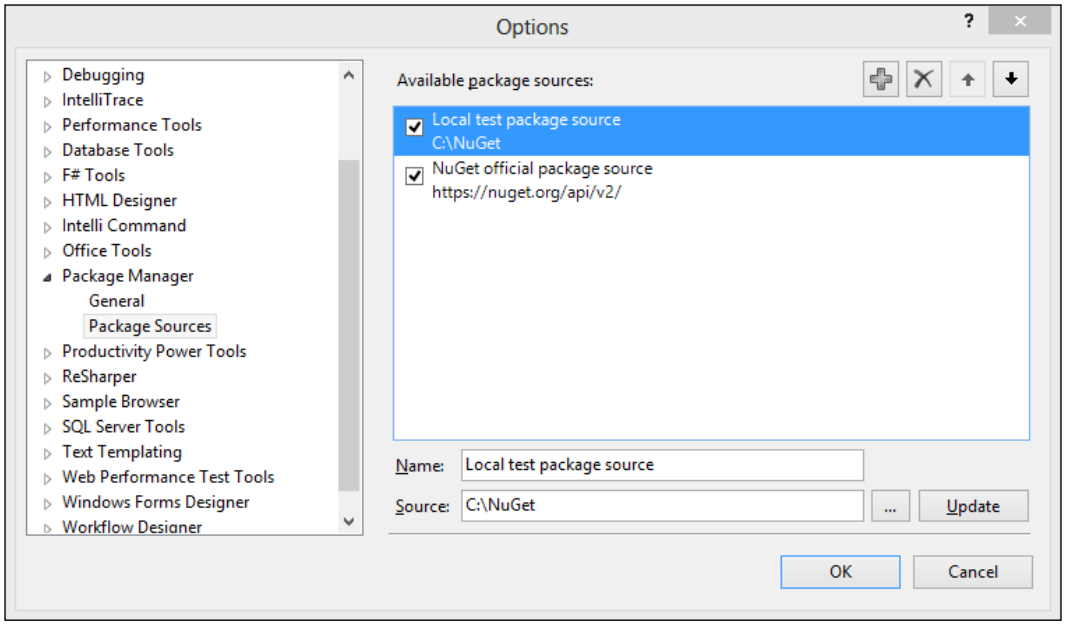
In this example, we have copied the files to the correct folders manually at the end of the build, and we will continue to do so throughout the book. In the real world, the files will usually be copied by the build script to the right place as a part of the build process. No human intervention will be required to build the NuGet package.

Testing a NuGet package locally

Before moving on, we should try installing the newly created package into projects with different target platforms to make sure it works as expected. By default, the NuGet Package Manager only searches for the packages online, but we can add other sources in its settings:

1. It's probably best to create a single local folder (for example, `C:\NuGet`) and copy all the packages for local testing into it. Let's do it now and copy the newly created package there.
2. Click on the **Package Manager Settings** menu item by navigating to **Tools | Library Package Manager** to open the Visual Studio **Options** dialog box.
3. Click on the **Package Sources** tree node in the tree view on the left to edit the NuGet package sources.
4. Click on the "plus" button in the top right of the dialog to add a new package source.
5. Set its **Name** value to, say, `Local test package source`.
6. Set its **Source** value to the folder created in the first step. You can use a folder dialog box if you click on the **...** button on the right of the textbox.
7. Click on the **Update** button to confirm the changes.

8. Use the "up arrow" button in the top right of the dialog box to move the new package source to the top of the list. This way, NuGet will first search this package source if you haven't selected it explicitly.



With the local test package source set up, we can test the package that we have previously created. Let's try installing it in different types of projects and see what happens:

- Create a new project based on the **Class Library** template targeting .NET Framework 4. After you have installed the package, you can check if the project references the .NET 4 version of the assembly.
- Create a new project based on the **Class Library** template targeting .NET Framework 3.5. This time the project will reference .NET 2.0 version of the assembly, once you have installed the package, as the best match out of the available assemblies.
- Create a new project based on the **Silverlight Class Library** template targeting any version of Silverlight. When you attempt to install the package, the process will fail because there is no compatible version of the assembly available in the package.

Folder naming conventions

As we have seen, the subfolder name within the `lib` folder determines the target platform of its contents. Of course, many more target platforms than the two that we have already used are supported.

The folder name is being parsed into a `FrameworkName` object, but it's easier to just remember the following conventions when naming the folders:

- When targeting the .NET Framework, the `net` prefix is used, followed by the minimum supported version, and an optional profile suffix starting with a hyphen: `net20`, `net35`, `net40-client`, and so on. Even if the `net` prefix is omitted, the .NET Framework is assumed, for example, `45`.
- When targeting Silverlight, the `sl` or `silverlight` prefix is followed by the minimum version: `sl3`, `sl4`, or `sl5`.
- When targeting Windows Phone, the `wp` or `windowsphone` prefix is followed by the minimum version: `wp7`, `wp71`, or `wp8`.
- When targeting Windows Store apps for Windows 8, `windows8` or `netcore45` is used.
- When targeting portable class libraries, the `portable` prefix is used, followed by a dash and a list of supported platforms separated by `+`: `portable-windows8+net45` or `portable-sl4+wp71+windows8`.

> Portable class libraries are special types of libraries that are binary compatible with multiple different frameworks. Inside them, a subset of the base class library (BCL classes), supported by the selected platforms, can be used. They were introduced as an extension to Visual Studio 2010 and are a part of Visual Studio 2012. You can learn more about them at `http://bit.ly/PclMsdn`.

The same set of subfolders can be created in the `content`, `build`, and `tools` package folders as well. Based on the target platform the package is being installed into, the contents of the best matching subfolder are used. The folders are matched in the following order:

1. The exact match of both the framework and the version.
2. The matching framework with the highest available version that is lower than the target. The backward compatibility of newer framework versions is being assumed.

3. The portable package with support for the same or lower versions of the matching framework.
4. The folder root, that is, files outside any subfolder; for example, those files that are directly inside the `lib` folder.

Only files from a single folder are used, that is, the best match according to the preceding algorithm, even if there are other files in the next match or root that are not present in the best match. For example, if there is only a single assembly in the folder exactly matching the target platform, that's the only one that will be referenced in the project even if there are other assemblies present in other matching subfolders or the `lib` folder.

Packaging localized satellite assemblies

When building localized solutions in .NET, resources for individual supported locales are linked to satellite assemblies that need to be distributed along with the main assembly. At runtime, they need to be placed into a subfolder beside the main assembly with a name matching the locale name of the resources. NuGet has support both for including the satellite assemblies in the same package as the main assembly and for creating individual packages for each supported locale.

Let's localize our sample library to create a set of satellite assemblies before we take a look at how to package them in both the ways supported by NuGet:

1. Add a new **Resources File** data file to the root of the project and name it `Messages.resx`. This one will contain the default strings in English.
2. Open it in Visual Studio and add a single row with the following values:
    1. In the **Name** column, enter `ExceptionEmptyQuery`.
    2. In the **Value** column, enter `A non-empty search query must be specified.`
3. Add another **Resources File** data file to the root of the project and name it `Messages.sl-SI.resx`. This one will contain the translated strings in Slovenian.
4. Open it in Visual Studio and add a single row with the following values:
    1. In the **Name** column, enter `ExceptionEmptyQuery`.
    2. In the **Value** column, enter `Podan mora biti neprazen poizvedbeni niz`.

5. Add the following lines of code at the beginning of the `Query` method (assuming the default namespace of your project is `Library`):

```
if (String.IsNullOrWhiteSpace(searchQuery))
{
    var resourceManager = new ResourceManager(
        "Library.Messages",
        typeof(GoogleSearchApi).Assembly);
    throw new ArgumentOutOfRangeException("searchQuery",
        resourceManager.GetString("ExceptionEmptyQuery"));
}
```

6. Add the following `using` statement at the top of your source file:

```
using System.Resources;
```

7. Build the project and make sure that there is a subfolder in the output folder named `sl-SI` that contains a satellite assembly named `Library.resources.dll` (assuming the main assembly is named `Library.dll`).

We now have everything we need to create localized NuGet packages.

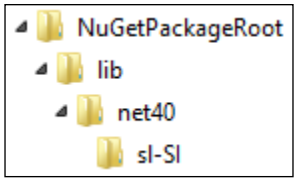
Adding localized assemblies to the main package

We need to include two files in the package:

1. Copy the main `Library.dll` assembly file into `lib\net40 folder`.
2. Copy the satellite `Library.resource.dll` assembly file into the `sl-SI` subfolder of `lib\net40`, just as it was created when building the project.

For the sake of simplicity, we have included only the files for the .NET Framework 4.0 in the package. To support multiple target platforms, the satellite assemblies need to be copied to each `lib` subfolder in the same way.

We should now have the following folder structure:



This time there's no need to change the `nuspec` file in any way. We'll just create the package by executing the following command line:

```
C:\SolutionDir> NuGet.exe pack GoogleSearchApi.nuspec -BasePath
NuGetPackageRoot
```

Before continuing, we need to copy the generated package to the previously created source folder of the local testing package so that we can test it:

1. Create a new project based on the **Console Application** template targeting .NET Framework 4.0.
2. Install the newly created package into the project.
3. Implement the `Program` class as follows:

```
using System;
using System.Globalization;
using System.Threading;
using Library;

namespace Application
{
    class Program
    {
        static void Main(string[] args)
        {
            CatchLocalizedException("en-US");
            CatchLocalizedException("sl-SI");
            Console.ReadLine();
        }

        static void CatchLocalizedException(string culture)
        {
            Thread.CurrentThread.CurrentUICulture =
                new CultureInfo(culture);
            try
            {
                GoogleSearchApi.Query(null);
            }
            catch (Exception e)
            {
                Console.WriteLine(e.Message);
            }
        }
    }
}
```

4. Run the application. It should write both the English and localized exception message to the console, proving that the localization satellite assemblies were correctly distributed and installed with the package:

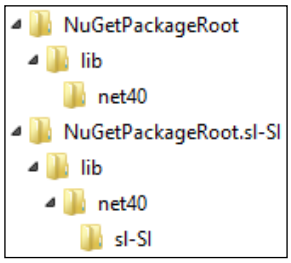
```
A non-empty search query must be specified.
Parameter name: searchQuery
Podan mora biti neprazen poizvedbeni niz.
Parameter name: searchQuery
```

The main advantage of this approach is its simplicity, but it comes with the price of a potentially larger package if there are many localized resources, and the requirement to release a new version of the complete package even if only an existing localization is fixed or a new one is added.

Creating satellite packages

When creating satellite packages, the main package only contains the main assembly as if it wasn't localized at all. For each supported locale, a separate satellite package needs to be created. It only contains the satellite assembly in the locale-specific subfolder, that is, at the same location as it was placed when it was included in the main package. There's no main assembly in the satellite package.

The folder structure for both the packages should be as follows:



While the `nuspec` file for the main package still doesn't need to be changed at all, the `nuspec` file for the satellite assemblies requires several modifications in comparison to it:

- The `id` element value must be appended with the locale name's postfix; the root of the package name must match the main package ID exactly:

  ```
  <id>GoogleSearchApi.sl-SI</id>
  ```

- A `language` element needs to be added. The locale names in the language element, the package ID, the subfolder name, and the package name must match exactly:

  ```
  <language>sl-SI</language>
  ```

- All `dependencies` of the main package must be removed and a dependency must be added on the main package instead. The brackets around the version number are required, specifying that the main package version must be an exact match:

```
<dependencies>
  <dependency id="GoogleSearchApi" version="[1.0.0]" />
</dependencies>
```

All of these changes are required for the NuGet Package Manager to recognize the satellite package as such. In this case, it copies the satellite package contents to the main package contents inside the `Packages` folder during installation, which is a prerequisite for Visual Studio to recognize the satellite assemblies and copy them to the output folder correctly during the build so that the localization actually works.

Additionally the `title`, `summary`, and `description` element values can be translated into the target locale to create a localized experience when browsing the packages in the **Manage NuGet Packages** dialog box.

Once everything is set up correctly, we can create both the packages by executing the following two calls at the command line:

```
C:\SolutionDir> NuGet.exe pack GoogleSearchApi.nuspec -BasePath
NuGetPackageRoot
```

```
C:\SolutionDir> NuGet.exe pack GoogleSearchApi.sl-SI.nuspec -BasePath
NuGetPackageRoot.sl-SI
```

Copy the two generated packages to the source folder of the local testing package, and you're ready for testing. We can use the same test scenario as for the single package case with the only difference that the `GoogleSearchApi.sl-SI` package needs to be installed instead of `GoogleSearchApi`.

Apart from checking the output of the application, you can also check the contents of `packages\GoogleSearchApi.1.0.0`. Inside the `lib\net40` subfolder, you should find the `sl-SI` folder containing the `Library.resource.dll` file from the `GoogleSearchApi.sl-SI` package copied there by NuGet because it recognized the package to be a satellite package.

The main advantage of having satellite packages is that you can ship them independently of the main package; this also results in your having a smaller main package with a smaller total size, since usually only a selection of locale files is actually installed. The price to pay is the additional complexity of having multiple packages with very strict conventions. Maintenance becomes more complex as well; since satellite packages require a strict match for the main package version, a new version of the main package requires new versions of all satellite packages with an updated main package dependency, even if their contents didn't change at all.

Adding new files and modifying existing ones

Although that's their main feature, NuGet packages are not limited to adding assembly references and their dependencies to the project. It's time to look at other types of files that can be added to a project when a package is installed.

Transforming configuration files

Let's start with configuration files. Depending on the type of project (desktop or web application), usually either `app.config` or `web.config` is included in it. Since these files contain different configuration options for the application, it's not unusual for a package to need to add some configuration options of its own:

This can be achieved by adding an `app.config.transform` and/or a `web.config.transform` file to the `content` folder of the package. We'll add such a file to our package from *Chapter 3*, *Creating and Publishing a Package*, to see how it works.

1. First, prepare a `NuGetPackageRoot` folder with the library assembly in the `lib\net40` subfolder if you don't have it handy from the previous examples.
2. Place an `app.config.transform` file with the following contents in the `content` subfolder of `NuGetPackageRoot`:

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <appSettings>
    <add key="ConflictingValue" value="NewValue" />
    <add key="MissingValue" value="NewValue" />
  </appSettings>
  <system.diagnostics>
    <trace autoflush="true">
      <listeners>
        <add name="newListener"
          type="System.Diagnostics.TextWriterTraceListener"
          initializeData="Trace.log" />
      </listeners>
    </trace>
  </system.diagnostics>
</configuration>
```

3. Create a new package using the following command line and place it in the source folder of the local testing package:

```
C:\SolutionDir> NuGet.exe pack GoogleSearchApi.nuspec -BasePath
NuGetPackageRoot
```

4. Create a new project in Visual Studio based on the **Console Application** template, targeting .NET Framework 4.
5. Make sure there's no `app.config` file in the project (delete it in case there is one), and install the just created NuGet package. You will notice an `app.config` file being added to the project, exactly matching `app.config.transform` from the package.
6. Now uninstall the package from the project. If you look at `app.config`, you'll see that it remained in the project, with everything but the root `configuration` element removed from it.
7. Manually modify the `app.config` file in the test project to match the following:

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <appSettings>
    <add key="ConflictingValue" value="OldValue" />
  </appSettings>
  <system.diagnostics>
    <trace>
      <listeners />
    </trace>
  </system.diagnostics>
</configuration>
```

8. Once again, install the package into the project. Once it's done, `app.config` should look as follows (except for the changes in formatting):

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <appSettings>
    <add key="ConflictingValue" value="OldValue" />
    <add key="ConflictingValue" value="NewValue" />
    <add key="MissingValue" value="NewValue" />
  </appSettings>
  <system.diagnostics>
    <trace autoflush="true">
      <listeners>
        <add name="newListener"
          type="System.Diagnostics.TextWriterTraceListener"
          initializeData="Trace.log" />
      </listeners>
    </trace>
  </system.diagnostics>
</configuration>
```

As you can see, both files got merged together. Let's take a closer look at the rules used and how they apply to the highlighted lines:

- If an element was not present in the target file before, it gets added. This happened with the `add` element inside the `listeners` element.
- When an element is present and doesn't have any attributes or all attribute values match the values in the transformation file, additional attributes are added to it. This happened with the `trace` element.
- When an element is present but at least one attribute value doesn't match, another element gets added to the target file. Both `add` elements inside the `appSettings` element are an example of this.

When such a package is uninstalled, all elements from the transformation file that have an exact match in the target will be removed. If we delete the old entry in the `appSettings` element before uninstalling the package, we will once again get a completely empty configuration file, because from NuGet's point of view, `app.config` exactly matched the transform file; therefore, all the entries must have been added this way.

The XML-Document-Transform syntax

Obviously, transforming configuration files this way has many limitations; therefore, some caution should be used when taking advantage of it to avoid unpleasant surprises for the consumer when the package is installed.

Since NuGet 2.6, an alternative, more flexible approach for transforming configuration files is available, namely, the **XML-Document-Transform** (**XDT**) syntax, as we know it from the `web.config` transformations when deploying web projects in Visual Studio. To use it, instead of adding the `app.config.transform` file into the `content` folder, two files need to be added:

- `App.config.install.xdt` will be applied to the `app.config` file when a package is installed in the project
- `App.config.uninstall.xdt` will be applied to the `app.config` file when a package is uninstalled from the project

The XDT syntax adds two attributes to elements, defining what will happen when the transformation is applied:

- The `xdt:Locator` attribute specifies how to find an existing matching element in the target file. It supports three operators: `Condition`, `Match`, and `XPath`. When omitted, the element is matched with another element in the target file based on its own `XPath`.

- The `xdt:Transform` attribute specifies how to modify the existing element once it is matched. Its supported values are `Replace`, `Insert`, `InsertBefore`, `InsertAfter`, `Remove`, `RemoveAll`, `RemoveAttributes`, and `SetAttributes`.

To modify the value of an existing entry in `appSettings`, the following transform file can be used:

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration xmlns:xdt="http://schemas.microsoft.com/XML-
  Document-Transform">
  <appSettings>
    <add key="ConflictingValue" value="NewValue"
        xdt:Locator="Match(key)" xdt:Transform="SetAttributes" />
  </appSettings>
</configuration>
```

A detailed explanation of the XDT syntax is not within the scope of this book. A good starting point to get to know more about it is the MSDN article at `http://bit.ly/XdtSyntax`.

Adding source code files

Any file included in the `content` folder of a NuGet package will automatically be added to the project when the package is installed. The relative file path inside the `content` folder will be preserved in relation to the project root when the file is copied to the project. The only exceptions to this rule are special files recognized by NuGet that are processed in a different way. XML transformation files with `.transform` and `.xdt` extensions are examples of such files.

Let's modify our existing example in such a way that instead of distributing a compiled assembly, we will distribute its source code and include it in the target projects when the package is installed:

1. Create a brand new `NuGetPackageRoot` folder and copy the unmodified `GoogleSearchApi.cs` source file from your sample library into `content\Library`.
2. Without modifying the `nuspec` file in any way, create a new package using the following command line and place it in the source folder of the local testing package:

   ```
   C:\SolutionDir> NuGet.exe pack GoogleSearchApi.nuspec -BasePath
   NuGetPackageRoot
   ```

3. Create a new project based on the **Console Application** template targeting .NET Framework 4, and install the package. Note how when the `Library` folder gets created in the project, the source file is copied into it and included in the project to be compiled. Also, the `Newtonsoft.Json` dependency package still gets installed and referenced correctly.
4. Since the included source file doesn't match the default namespace of the console application project, modify it manually.
5. Try uninstalling the package. It will succeed, but it won't remove the source file because it has been changed. The uninstall process doesn't fail because of it; everything except the changed files gets removed.
6. Install the package once again, without deleting the file that was left behind. This time, NuGet asks what needs to be done with the conflicting file in the folder. The same will happen when the package is being updated if the file is a modified one. In this case, allow it to be overwritten.

As long as there is no need to modify the files added to the project by a NuGet package, the installing, uninstalling, and updating of the packages happens flawlessly. If any of the files are changed, updating them becomes a problem because NuGet can't merge the changes; it can only overwrite the files or leave them unchanged.

Modifying the included files should therefore be avoided if at all possible. Making the classes partial so they can be extended in another file is usually good practice. Sometimes, modifications can also be avoided by transforming the source files before including them in the project. We'll take a look at that next.

Visual Studio project properties

In the previous example, we manually modified the namespace of the included file to match the target project's default namespace. It turned out this wasn't a good idea since it broke the package update process. Also, this can't be done using partial classes as suggested in the previous section. Fortunately, there is another way to modify the source files when they are installed:

1. Rename the file `GoogleSearchApi.cs` in the `content\Library` subfolder of `NuGetPackageRoot` from the previous example to `GoogleSearchApi.cs.pp`.
2. Open the file and modify the namespace declaration by adding the `$rootnamespace$.` prefix to the namespace declaration:

   ```
   namespace $rootnamespace$.Library
   ```

3. Create the package with the modified file using the following command line and place it in the source folder of the local testing package:

   ```
   C:\SolutionDir> NuGet.exe pack GoogleSearchApi.nuspec -BasePath
   NuGetPackageRoot
   ```

4. Install the new package into the test project from the previous example. Open the `GoogleSearchApi.cs` file that was added to the project and note how the `$rootnamespace$` placeholder has been replaced with the project's default namespace.

Any file with a `.pp` extension in the `content` folder gets preprocessed by NuGet before it is added to the project:

- Placeholders (starting and ending with the `$` character, for example, `$rootnamespace$`) are replaced with the value of a corresponding Visual Studio project property. In the preceding example, `RootNamespace` was such a property.

  You can find a complete list of Visual Studio project properties at `http://bit.ly/VsProperties`.

- The `.pp` extension is removed from the filename.

Adding MSBuild targets and properties

The only way to actually affect the project build process with a NuGet package is to modify the project file, which is just an MSBuild file with a different extension. With NuGet 2.5, this became much simpler. In most cases, it's enough to include one or two MSBuild files in the package; everything else is taken care of by NuGet.

The included MSBuild files must be named as follows (in both cases, `PackageId` should be replaced with the actual ID of the package):

- `PackageId.props` should contain properties and will be included at the beginning of the project file
- `PackageId.targets` should contain build targets and will be included at the end of the project file

Let's take advantage of this feature and extend our sample package to display a thank you message at the end of the build.

> Learning MSBuild is not within the scope of this book. You can find a short tutorial at `http://bit.ly/MSBuildWalkthrough`.

1. Create a new MSBuild file named `GoogleSearchApi.props` with the following contents:

```
<?xml version="1.0" encoding="utf-8" ?>
<Project ToolsVersion="4.0" xmlns=
  "http://schemas.microsoft.com/developer/msbuild/2003">
  <PropertyGroup>
    <NugetEssentialsThanks>Thank you for reading NuGet 2
        Essentials.</NugetEssentialsThanks>
  </PropertyGroup>
</Project>
```

2. Create another MSBuild file named `GoogleSearchApi.targets` with the following contents:

```
<?xml version="1.0" encoding="utf-8" ?>
<Project ToolsVersion="4.0" xmlns=
  "http://schemas.microsoft.com/developer/msbuild/2003">
  <Target Name="AfterBuild">
    <Message Text="$(NugetEssentialsThanks)"
             Importance="high" />
  </Target>
</Project>
```

3. Copy both files to the `build` subfolder of `NuGetPackageRoot`.
4. Create the package using the following command line and place it in the source folder of the local testing package:

```
C:\SolutionDir> NuGet.exe pack GoogleSearchApi.nuspec -BasePath
NuGetPackageRoot
```

5. Create a new project based on the **Console Application** template targeting .NET Framework 4, and install the package.
6. After building the project, open the **Output** window (navigate to **View | Output** or press *Ctrl* + *W* and then *O*). You should see the following text as the last output of the project build:

```
Thank you for reading NuGet 2 Essentials.
```

7. Right-click on the project in **Solution Explorer** and select **Unload Project** from the context menu. Right-click on it again and select **Edit ProjectName.csproj** (where **ProjectName** is the name of your project) to open the project file for editing.
8. Find the two modifications made by NuGet to the project file. Just below the root element, `GoogleSearchApi.props` is imported:

   ```
   <Import Project="..\packages\GoogleSearchApi.1.0.0\build\
   GoogleSearchApi.props" Condition="Exists('..\packages\
   GoogleSearchApi.1.0.0\build\GoogleSearchApi.props')" />
   ```

   At the end of the file, above the root element's closing tag and any comments, `GoogleSearchApi.targets` is imported:

   ```
   <Import Project="..\packages\GoogleSearchApi.1.0.0\build\
   GoogleSearchApi.targets" Condition="Exists('..\packages\
   GoogleSearchApi.1.0.0\build\GoogleSearchApi.targets')" />
   ```

Both imports are safe; they check that the file actually exists, allowing the project to load even if the packages aren't restored and the files aren't available.

Including PowerShell scripts in a package

PowerShell scripts are the last resort when none of the features described so far can be used to achieve the desired goal. Let's see what the available options for using them are.

> To learn more about PowerShell, visit `http://bit.ly/PSGetStarted`.

PowerShell scripts must be placed inside the `tools` subfolder. NuGet automatically invokes the following scripts based on their name:

- `Init.ps1` is run the first time the package is installed into a solution and every time the solution is opened. Unlike other `ps1` files, it shouldn't be put into a framework-specific folder since it's not tied to a specific project; otherwise, it will be ignored.
- `Install.ps1` is run every time the package is installed into a project. `Init.ps1` is run before it, when applicable.
- `Uninstall.ps1` is run every time the package is uninstalled.

Each of these scripts should have the following line at the top, allowing NuGet to pass the parameters:

```
param($installPath, $toolsPath, $package, $project)
```

The parameters will have the following values:

- `$installPath` will contain the path to the folder where the package is installed
- `$toolsPath` will contain the path to the `tools` folder in the package; this is where the script is run from
- `$package` will contain a reference to the package object
- `$project` will contain a reference to the DTE project object into which the package is installed; it will be empty in `Init.ps1` because it is run at the solution level and not for a specific project

The **Development Tools Environment** (**DTE**) project object is part of Visual Studio automation and the extensibility API. You can find out more about it on its reference page at `http://bit.ly/VsDteProject`.

We're going to take a look at two examples of PowerShell script usage in NuGet to get you started. Examining existing packages is the best way to learn more, though. Once a package is installed into a project, its contents are placed into a subfolder inside the `packages` folder at the solution level. All the PowerShell scripts can be found in the `tools` folder and can be opened with any text editor.

Running a script at install time

The first example will open an HTML page after the package is installed into the project:

1. Create a file named `Install.ps1` with the following contents:

```
param($installPath, $toolsPath, $package, $project)
  $path = Join-Path $toolsPath "ReleaseNotes.html"
  $dte.ItemOperations.Navigate($path)
```

2. Create a file named `ReleaseNotes.html` with the following contents:

```
<!DOCTYPE html>
<html lang="en" xmlns="http://www.w3.org/1999/xhtml">
<head>
    <meta charset="utf-8" />
    <title>Release Notes</title>
</head>
```

```
<body>
    <h1>Release Notes</h1>
    <p>Thank you for reading NuGet 2 Essentials!</p>
</body>
</html>
```

3. Place both files in the `tools` subfolder inside `NuGetPackageRoot` and call the following command line to create the package:

   ```
   C:\SolutionDir> NuGet.exe pack GoogleSearchApi.nuspec –BasePath
   NuGetPackageRoot
   ```

4. Copy the package to the source folder of the local testing package, create a new project based on the **Console Application** template targeting .NET Framework 4, and install the package.

Once the installation is complete, the `ReleaseNotes.html` file will be opened inside Visual Studio.

Adding a PowerShell module

In the second example, we will create a simple PowerShell module with an additional command that can be called from the **Package Management Console** window:

1. Create a module file named `module.psm1` with the following contents:

   ```
   Function Show-Thanks
   {
     Write-Host "Thank you for reading NuGet 2 Essentials!"
   }
   Export-ModuleMember -Function Show-Thanks
   ```

2. Create a script file named `init.ps1` with the following contents:

   ```
   param($installPath, $toolsPath, $package)
     $path = Join-Path $toolsPath module.psm1
     Import-Module $path
   ```

3. Place both files in the `tools` subfolder inside `NuGetPackageRoot` and call the following command line to create the package:

   ```
   C:\SolutionDir> NuGet.exe pack GoogleSearchApi.nuspec –BasePath
   NuGetPackageRoot
   ```

4. Copy the package to the source folder of the local testing package, create a new project based on the **Console Application** template targeting .NET Framework 4, and install the package.
5. Open the **Package Management Console** window by navigating to **Tools | Library Package Manager | Package Manager Console** and invoke the following command:

   ```
   PM> Show-Thanks
   ```

The following text will be the output, as defined in our module:

```
Thank you for reading NuGet 2 Essentials!
```

The same technique can be used to include modules of any complexity. The `Init.ps1` script will make sure the commands are available whenever a solution with this installed package is loaded, even after we restart Visual Studio.

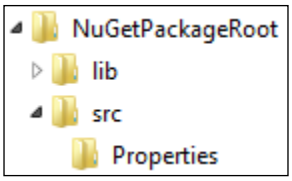
Debugging of packages

There's one last topic to address, that is, how to enable debugging of our library by uploading its sources to a symbol server. For this purpose, NuGet is integrated with `http://www.symbolsource.org`; this provides an easy workflow for both publishing the sources and downloading them when debugging. Let's take a look at both.

Creating and publishing a symbol package

A symbol package must contain all the assembly files (with a `dll` extension), their corresponding symbol files (with a `pdb` extension), and of course, the sources, but none of the other files that could be included in the actual NuGet package as they were discussed before. Fortunately, NuGet will automatically include the right files in each package even if we put all of them in a single folder structure, so long as we provide the `Symbols` option instructing it to create a symbol package at the same time as the NuGet package.

1. As in all of the previous examples, start by building a sample `GoogleSearchApi` library.
2. Copy both the resulting `Library.dll` and `Library.pdb` files to the `lib\net40` subfolder of `NuGetPackageRoot`.
3. Create an `src` subfolder inside `NuGetPackageRoot`.
4. Copy all of the library source files to the newly created subfolder, preserving their relative paths; that is, copy only the `cs` and `csproj` files of the `Library` project folder to the `src` subfolder of `NuGetPackageRoot`. After this step, the folder structure should be as follows:



5. Since this time it will be necessary to publish the package to NuGet, you'll need to find a package ID that's still available, then change and rename the `nuspec` file accordingly.
6. Call the following command line to create the packages (note the additional `-Symbols` option at the end, and don't forget to change the `nuspec` filename according to your change from the previous step):

   ```
   C:\SolutionDir> NuGet.exe pack GoogleSearchApi.nuspec -BasePath
   NuGetPackageRoot -Symbols

   Attempting to build package from 'GoogleSearchApi.nuspec'.

   Successfully created package 'C:\SolutionDir\
   GoogleSearchApi.1.0.0.nupkg'.

   Successfully created package 'C:\SolutionDir\
   GoogleSearchApi.1.0.0.symbols.nupkg'.
   ```

7. If you check the contents of both the packages, you'll notice that the `nupkg` package still contains only the `Library.dll` file, while the `symbols.nupkg` file contains all the sources and the `Library.pdb` file as well.
8. To test how debugging will work for the consumers of your package, you'll need to publish both the generated packages. Copying them to the source folder of a local package is not a good enough test scenario in this case. Just make sure you have set your API key for NuGet as explained in *Chapter 3*, *Creating and Publishing a Package*, before calling the following command line (again, change the package name to match the one you have just generated):

   ```
   C:\SolutionDir> NuGet.exe push GoogleSearchApi.1.0.0.nupkg

   Pushing GoogleSearchApi 1.0.0 to the NuGet gallery (https://www.
   nuget.org)...

   Your package was pushed.

   Pushing GoogleSearchApi 1.0.0 to the symbol server (http://nuget.
   gw.symbolsource.org/Public/NuGet)...

   Your package was pushed.
   ```

9. Note how the symbol package was automatically pushed to the symbol server without creating an account there or setting up anything.

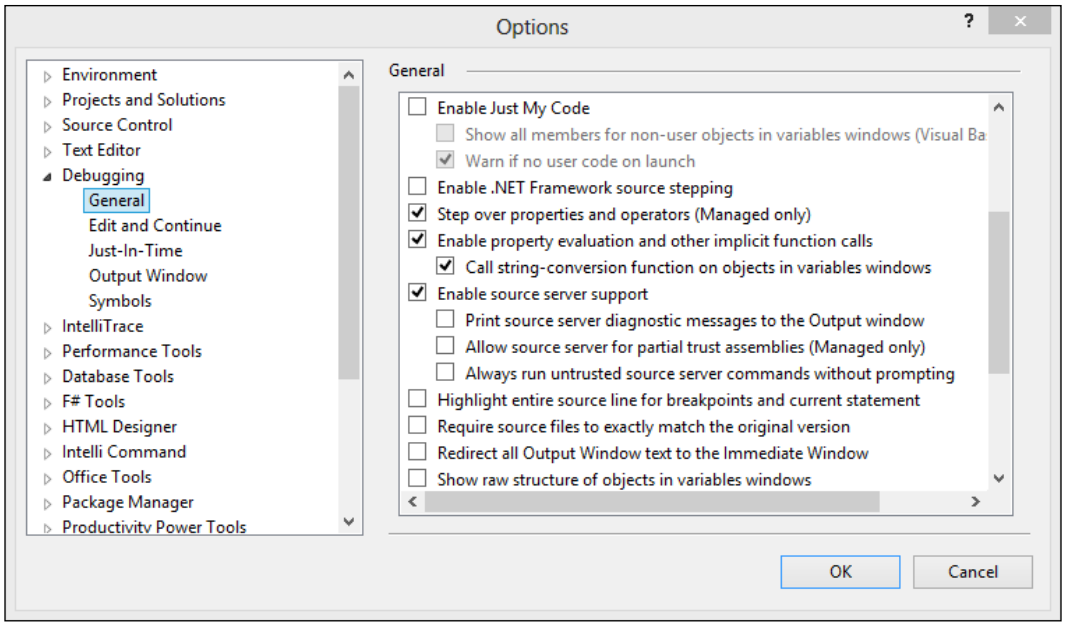
Although no account is required at `http://www.symbolsource.org` to publish a symbol package, it's a good idea to create one and associate it with your NuGet account if you plan on publishing symbol packages regularly, as it will allow you to manage them. To register for an account, visit the site and follow the instructions there.

Consuming a symbol package

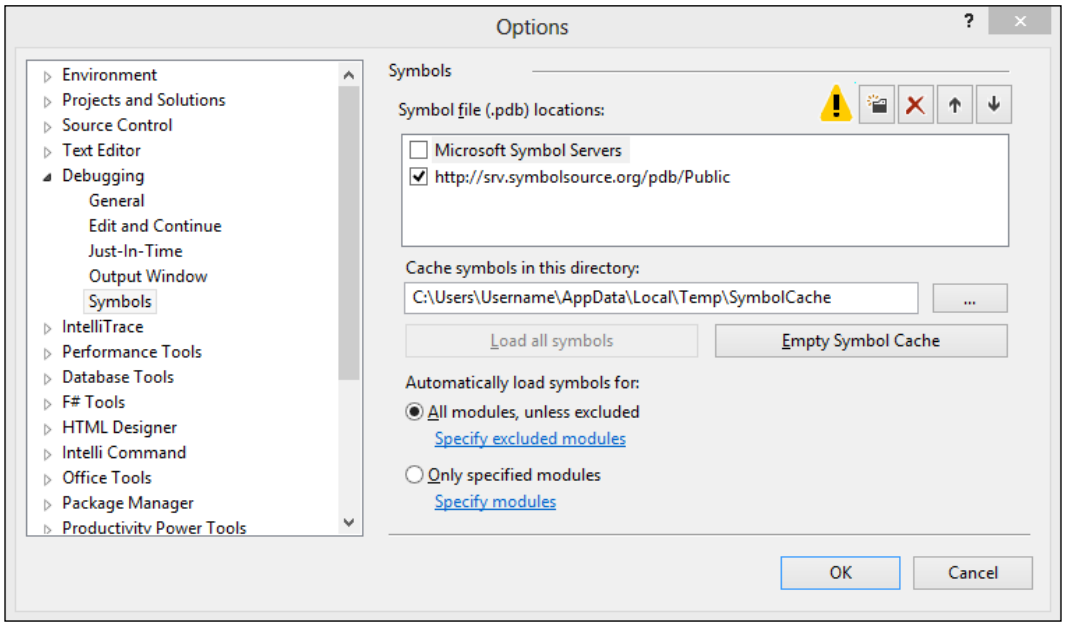
Once the symbol package is published on the symbol server, Visual Studio will automatically be able to download the sources when debugging. It just needs to be configured correctly beforehand.

Although the following steps should be enough for debugging to work, you should check the official instructions at `http://bit.ly/SymbolSourceVS` for complete and up-to-date information on setting up Visual Studio.

1. Click on the **Options…** menu item inside the **Tools** menu in Visual Studio to open the **Options** dialog box.
2. Navigate to **Debugging | General** in the tree view on the left-hand side of the dialog box.
3. Configure the following settings in the right-hand pane:
    1. Uncheck **Enable Just My Code**.
    2. Uncheck **Enable .NET Framework source stepping**.
    3. Check **Enable source server support**.
    4. Uncheck **Require source files to exactly match the original version**.



4. Navigate to **Debugging** | **Symbols** in the tree view.
5. Add `http://srv.symbolsource.org/pdb/Public` to the **Symbol file (.pdb) locations** listbox and enable it.



www.allitebooks.com

6. Create a new project based on the **Console Application** template targeting .NET Framework 4.0.
7. Install the previously published package into the project.
8. Implement the `Program` class as follows, and put a breakpoint at the highlighted line:

```
using Library;

namespace Application
{
    class Program
    {
        static void Main(string[] args)
        {
            var results = GoogleSearchApi.Query("NuGet");
        }
     }
}
```

9. Delete the library project you created the package from or move it to another location to make sure Visual Studio can't find it; otherwise, it will open the source files from the project instead of downloading them.
10. Run the application and wait for it to stop at the breakpoint.
11. Step into the method call by pressing *F11*.
12. Visual Studio should download and open the library source file, stopping at the first line of the `Query` method.
13. Hover over the source window tab to see the full file path. If you have previously removed the original library sources as instructed, the file should be located inside the `SymbolCache` directory; by default, it is set to `C:\Users\Username\AppData\Local\Temp\SymbolCache`.

The consumers only need to set up Visual Studio once to enable the debugging of any NuGet package with a published symbol package.

Summary

This chapter covered most of the NuGet features from the package creator's point of view. We started with a closer look at the NuGet package's folder structure. Based on this knowledge, we explained the creation of multiplatform and multilanguage packages. We continued with an exploration of the different types of files that can be included in a package to be added to the target project, such as XML file transformations, source files, and build files. Let's not forget PowerShell scripts; we discussed why one would include them in a package and tried the feature out with two different scripts. We concluded the chapter with a sample demonstration of how to enable debugging of packaged libraries for the consumers.

In the following chapter, we will learn about the different ways to set up our own NuGet server where we can host the packages we created internally.

5
Hosting an Internal NuGet Server

In previous chapters, we have covered all the fancy features that NuGet provides us with. We have created and published packages, mostly on the NuGet Gallery (`http://www.nuget.org/`). The official NuGet Gallery is a public feed, mostly used and focused for distribution of the open source libraries.

Not all development is public and open source, and maybe our packages are not meant for a larger audience. Maybe you own a company and you do not want your business knowhow to be out in the open. Then, we can host an internal NuGet feed and publish packages to our own repository; access to these packages and repositories is completely in our control. We can use the internal server to mirror packages from `nuget.org`, or just to have a private prerelease repository. There are a variety of options for setting up our repository as NuGet is very flexible. As mentioned in *Chapter 4*, *Adding Features to a NuGet Package*, we can simply use a folder in our local filesystem as a NuGet repository, which is acceptable for individual usage. But, if we want a more sophisticated repository, we can set up our own NuGet Gallery.

After reading this chapter, you will have all the necessary knowledge to choose what type of NuGet feed is best for your case and how to set up your own NuGet server.

Understanding reasons for hosting NuGet internally

There are plenty of reasons why we would want to host our internal feed. Since all packages on the official NuGet feed are public, we can't hide our development packages. In my experience, most of these packages are highly unstable and not ready to be published officially. With our own NuGet feed, we can publish packages to this feed and only people with access to this feed will have access to these packages.

Security plays a big role in corporate environments. You can set up a NuGet repository with your Active Directory or **Lightweight Directory Access Protocol** (**LDAP**) account, filtering users who have permission to publish or just to download the packages.

Common source control systems favor branch development, we can have one feed for each released branch. For example, one feed for the development branch, which is shared by all developers in team; one feed for the release-candidate branch; and one feed for the production packages and the production hotfix packages.

We can also use continuous integration systems to generate and publish packages in our feeds so that we can have automatically built, nightly build packages for all our components.

We might also want to use our internal feed just to mirror packages from the official NuGet feed; perhaps you want them on your local network where you can safely have a backup. After all, this is a public feed and packages can be deleted from the feed, leaving us with a missing dependency.

> There are a number of reasons why and how to host your own NuGet feed. The reasons listed here are the ones experienced by us at our employers and customers.

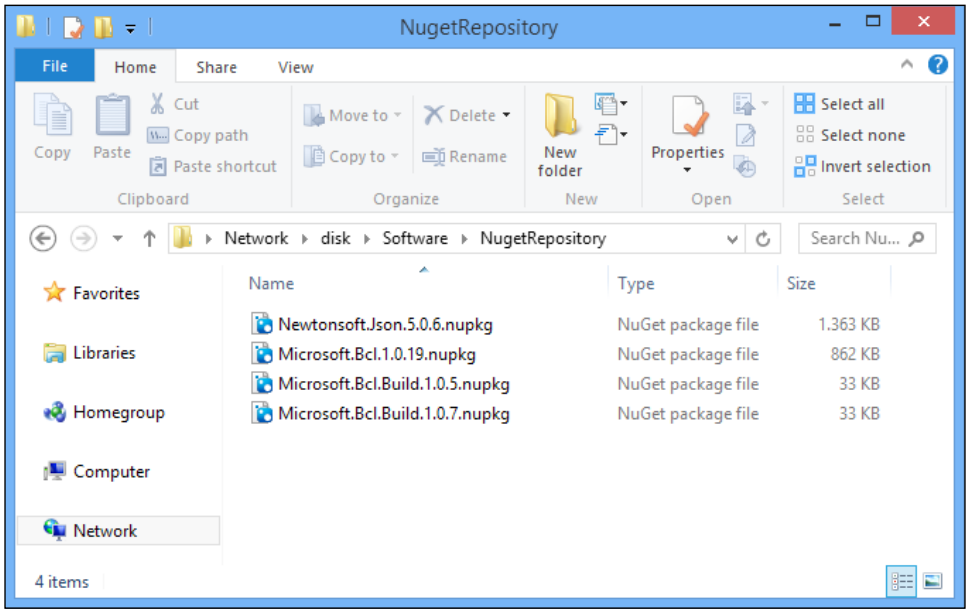
Creating a NuGet package repository

We can host a NuGet package repository in several ways. We're going to cover the most popular ones with a step-by-step tutorial on how to set up a NuGet package repository.

Creating a filesystem-based NuGet repository

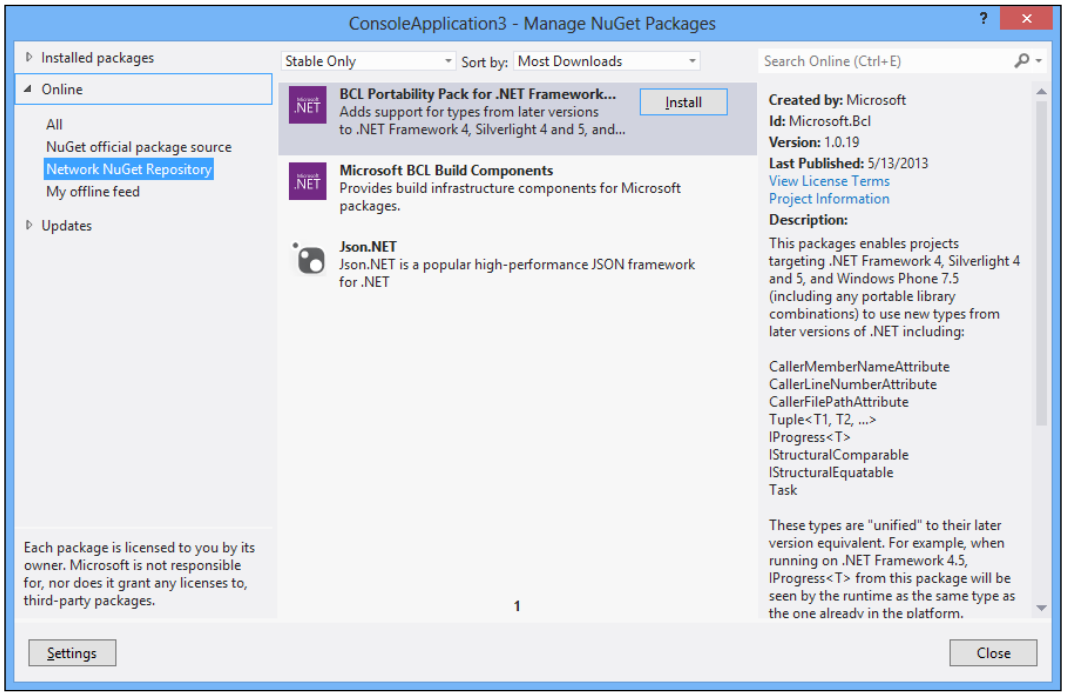
For individual usage, the most common way is to simply use a folder in a local filesystem. Since cloud storage is getting more and more popular, we can also use a folder created in a cloud storage provider (for example, SkyDrive, Dropbox, and so on) or just a shared folder on a local network. Publishers need to have write access to the shared folder. Using this method, you don't need an API key to push your packages; simply copying packages into the folder will show them in your feed.

A network share that acts like this is shown in the following screenshot:



In *Chapter 4*, *Adding Features to a NuGet Package*, we covered adding an additional NuGet feed into Visual Studio.

With the added `\\disk\Software\NuGetRepository` feed in VS, our feed should look like the following screenshot:



The NuGet cache

When a package is installed from the feed, it's also copied into the NuGet cache. NuGet caches all the retrieved packages into a local folder. By default, this folder is located in `%LOCALAPPDATA%\NuGet\Cache`.

Using this folder as a NuGet feed comes in handy when you are offline and you need to use previously used packages.

You can clear this cache by removing all packages from the folder or by using Visual Studio's **Options** menu, under **Package Manager** by selecting **Clear Package Cache**.

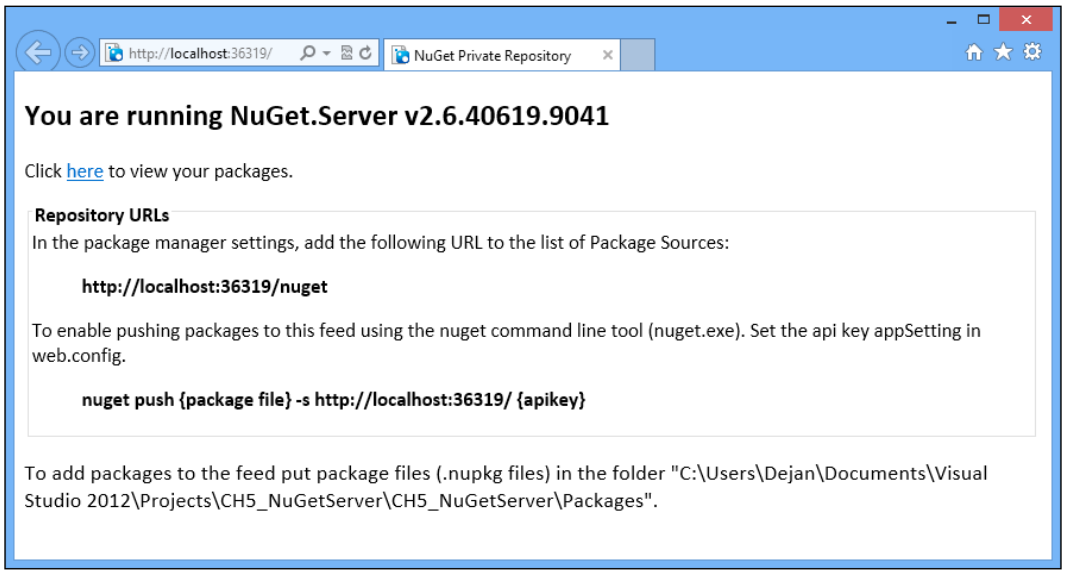
Creating a lightweight NuGet server

We can also create a very simple and lightweight NuGet server by using the `NuGet.Server` package from the NuGet Gallery. This package creates and sets up all the necessary components to get NuGet server up and working in a short time. To host this NuGet server, we will need the **Internet Information Services** (**IIS**).

Let's create a NuGet server in an empty ASP.NET web application.

1. In Visual Studio, navigate to **FILE | New | Project**.
2. In the left pane, expand the **Visual C#** node and select the **Web** node.
3. Select **ASP.NET Empty Web Application**.
4. Name the project as `CH5_NuGetServer`.
5. An empty web application is created containing only `web.config`.
6. Install the `NuGet.Server` package from the official NuGet Gallery.
7. Run the application by navigating to **Debug | Start Debugging** or press *F5*.

   A website similar to the following should appear in our web browser:



Configuring the NuGet server

Let's open the `web.config` file from the **Solution Explorer** window and navigate to the `appSettings` section in the XML file. The `NuGet.Server` package uses the configuration file transformation described in *Chapter 4*, *Adding Features to a NuGet Package* to add three settings to the configuration file: `requireApiKey`, `apiKey`, and `packagePath`.

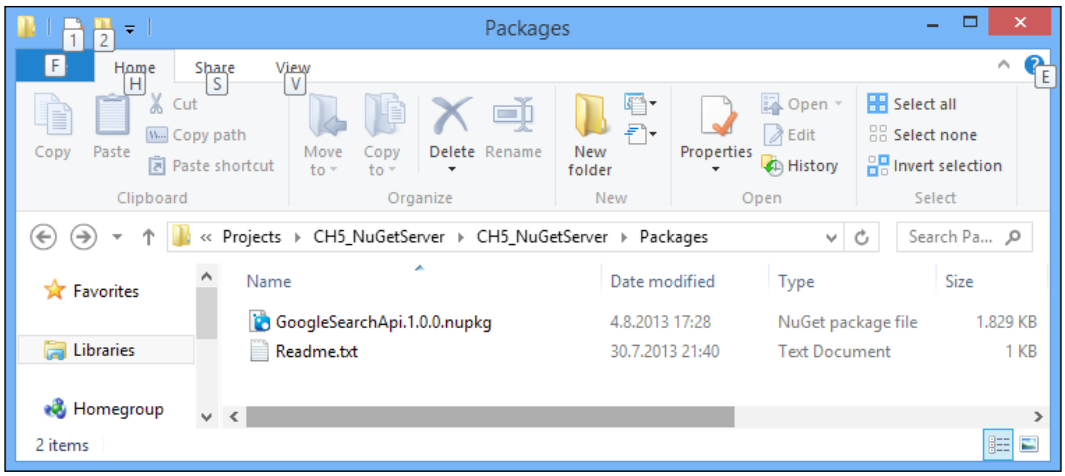
The following table describes these three settings:

Setting key	Description
`requireApiKey`	Boolean value; it determines whether an API key is required for pushing or deleting packages from this NuGet server.
`apiKey`	String value; it is a shared API key for all users. By default this is empty, which means that publishing is disabled.
`packagePath`	String value; it is the path to the `Packages` folder. This path can be physical or virtual. If left empty, the default path `~/Packages` is used.

Let's set the `apiKey` setting to `thisIsMySecretPassword` and publish the package created in the previous chapter in our newly created NuGet server.

```
D:\>nuget push GoogleSearchApi.1.0.0.nupkg thisIsMySecretPassword -s
  http://localhost:36319/
Pushing GoogleSearchApi 1.0.0 to 'http://localhost:36319/'...
Your package was pushed.
```

We didn't change our packages path, so the uploaded package should be in our `Packages` folder.

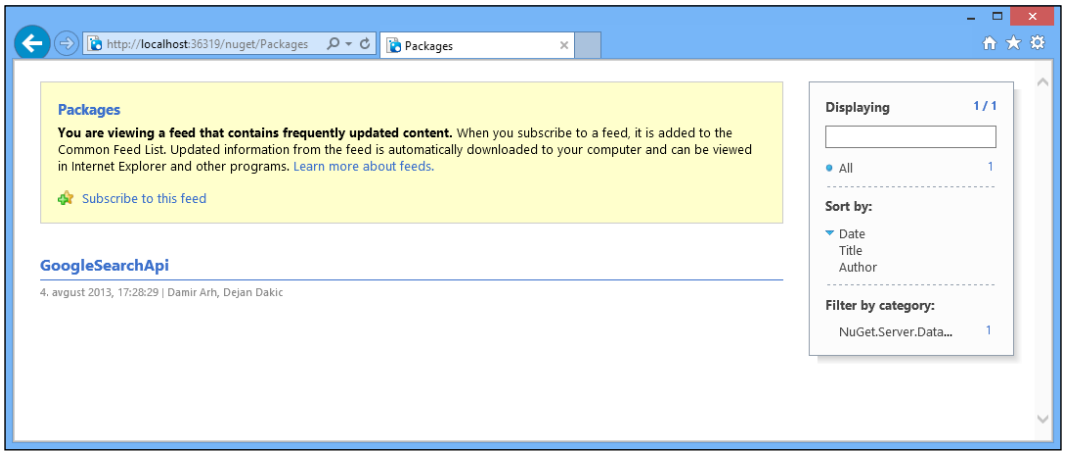


Accessing packages

Our NuGet server provides OData services for our feed. In this example, an OData service is available at `http://localhost:36319/nuget/Packages`.

> OData is a `REST`-based protocol created by Microsoft. You can learn more about the OData protocol at `http://bit.ly/intrOData`.

With the package uploaded, our OData service should look similar to the following figure. The right pane allows us to search, filter, and sort our packages.



We can also add the `http://localhost:36319/nuget` URL to the list of Package Source in Visual Studio's Package Manager.

Using a private feed with package restore

In *Chapter 2*, *Package Management*, we covered NuGet's package restore. When we're using packages from multiple sources, we need to add all package sources in the `NuGet.targets` file. This way, they are available for download during build or package restore.

We are going to add our private feed URL `http://localhost:36319/nuget` to the package sources.

Open the `NuGet.targets` file located in the `.nuget` folder. Search for an XML section similar to the following:

```
<ItemGroup Condition=" '$(PackageSources)' == '' ">
    <!-- Package sources used to restore packages. By default,
         registered sources under %APPDATA%\NuGet\NuGet.Config
         will be used -->
    <!-- The official NuGet package source
         (https://nuget.org/api/v2/) will be excluded if package
         sources are specified and it does not appear in the list
         -->
    <!--
        <PackageSource Include="https://nuget.org/api/v2/" />
        <PackageSource Include="https://my-nuget-source/nuget/" />
    -->
</ItemGroup>
```

As noted in the comment, we need to have the official NuGet package source added to the list if we're using packages from the official feed:

```
<ItemGroup Condition=" '$(PackageSources)' == '' ">
    <PackageSource Include="https://nuget.org/api/v2/" />
    <PackageSource Include="http://localhost:36319/nuget/ "/>
</ItemGroup>
```

Edit the `PackageSources` section, which will look like the preceding code. Note that we have added the official NuGet feed and our private feed.

During the package restore, NuGet will look up the package in the following order:

1. If a package with a specified version is available in the NuGet `Cache` folder, NuGet will use the package from the cache folder.
2. If a package is not found in the NuGet `Cache` folder, NuGet will try to retrieve the package from the official feed, which is the first package source in the `NuGet.targets` file.
3. If a package is not found in the official NuGet feed, NuGet will try to retrieve the package from the last package source listed: our private feed.

The NuGet Gallery

If the previous two repository types are not enough for us, there is always the NuGet Gallery. With NuGet Gallery, you can have the same site experience as you have on the official `www.nuget.org` site.

The following are a few of the features of the NuGet Gallery:

- With a user's unique API key, you can control user permissions
- Advanced statistics about your NuGet feed
- Advanced administration

In the upcoming sections, we will describe in a step-by-step fashion how to install and set up the NuGet Gallery. We will download and compile NuGet Gallery, configure Microsoft SQL Server, and then set up our first NuGet Gallery. We will also cover the basic administration of the NuGet Gallery.

The NuGet Gallery is an open source project available at `http://bit.ly/nugalgh`. The NuGet Gallery is a bit trickier to install and configure—but it's worth the pain.

Prerequisites

Before we can set up our NuGet Gallery, we need some software prerequisites. For compiling NuGet Gallery, we will use the following software, as described in the previous chapters also:

- Visual Studio 2010 or higher
- PowerShell 2.0 or higher

Installing the NuGet Gallery

The NuGet Gallery source code is stored on the GitHub hosting service, from which we should get the latest source code. The project's web page is available at `http://bit.ly/nugalgh`. The easiest way to get the source is by downloading the ZIP file containing the source code, which is available at `http://bit.ly/nugalzip`. We could also use the Git client, but for the sake of simplicity we will use the ZIP archive.

For now, just download the ZIP from the mentioned URL and extract it into a desired folder; in this example, we will use `C:\NuGetGallery`.

Before we start changing our configuration file, we must set up database access for our NuGet Gallery.

Configuring and compiling the NuGet Gallery

For demonstration purposes, we won't use SSL. We'll use the `LocalDB` database as the data source. We will have to edit some code to meet these requirements.

Navigate to the folder with the extracted ZIP file contents, (`C:\NuGetGallery` in this example) and then open the solution file named `NuGetGallery.sln`.

1. In the **Solution Explorer** window, open the `web.config` configuration file.
2. Navigate to the `appSetting` section and set `Gallery.GalleryOwner` to your e-mail address.
3. In the `appSettings` section, change `Gallery.SiteRoot` to your NuGet Gallery's hostname.
4. Save the changes and open Command Prompt and navigate to the folder with the extracted ZIP file. Then run the `build.bat` script file.

   ```
   C:\NuGetGallery>build.bat
   Microsoft (R) Build Engine version 4.0.30319.17929
   [Microsoft .NET Framework, version 4.0.30319.18051]
   Copyright (C) Microsoft Corporation. All rights reserved.

     Installing 'AnglicanGeek.MarkdownMailer 1.2'.
     Installing 'AnglicanGeek.WindowsAzure.ServiceRuntime 1.6'.
     Successfully installed 'AnglicanGeek.MarkdownMailer 1.2'.
     Successfully installed
       'AnglicanGeek.WindowsAzure.ServiceRuntime 1.6'.
   ...
   ```

   The preceding script will download the required packages using NuGet of course, compile our NuGet Gallery, and run tests. The compiled application is stored in the `_PackagedWebsite` directory.

Now we have to create the database for our NuGet Gallery, so let's switch back to Visual Studio.

The Website project should be set as the startup project.

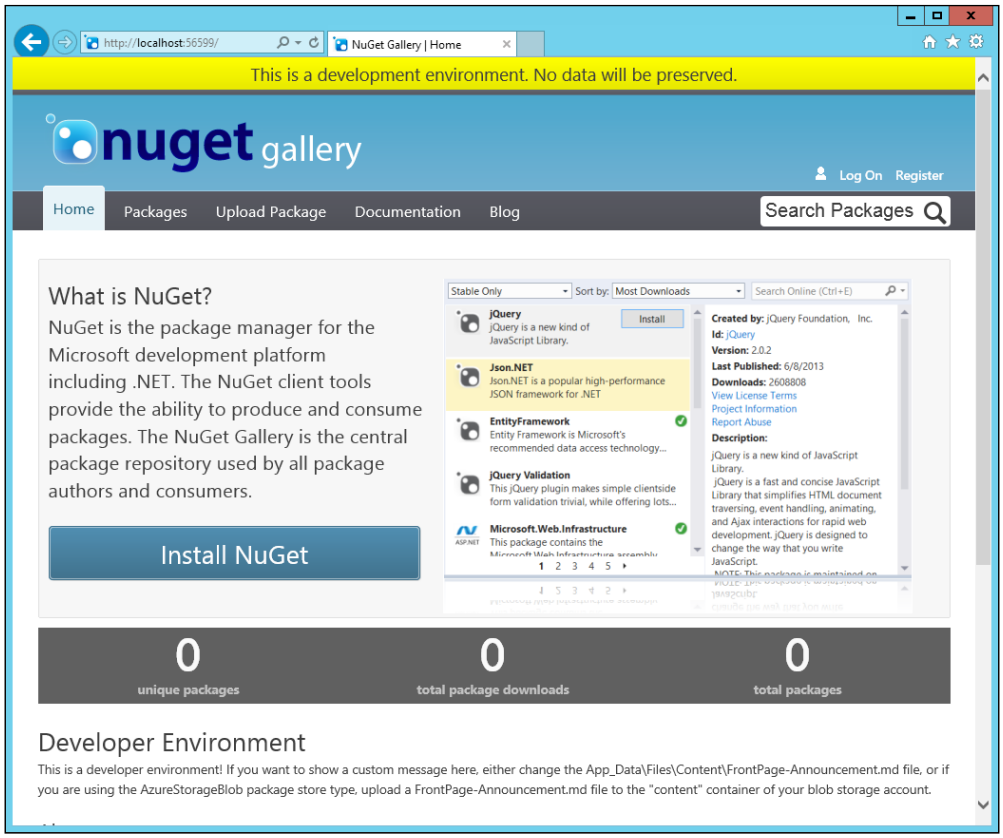
- If **Website** isn't set as the startup project, we can accomplish that by right-clicking on the **Website** project and selecting **Set as StartUp Project** from the context menu.
- Use **Package Manager Console** to create the database required for NuGet Gallery to run. This can be achieved with the `Update-Database` command. Make sure you have selected **Website** as **Default project** in **Package Manager Console**. Updating of the database should start with an output similar to the following:

  ```
  PM> Update-Database
  Specify the '-Verbose' flag to view the SQL statements being
  applied to the target database.
  ```

```
Applying code-based migrations: [201110060711357_Initial,
201110102157002_PrereleaseChanges, 201110180052097_
GallerySettings, 201110230649210_PackageOwnerRequests,
201111022024584_PackageDependencyVersionSpec, 201111022051010_
PackageReleaseNotes, 201111080239544_ListPackagesIndexes,
201111080816426_DisplayPackageIndexes, 201111081908453_
MyPackagesIndexes, 201111150729167_AddSmt...].

Applying code-based migration: 201110060711357_Initial.

Applying code-based migration: 201110102157002_PrereleaseChanges.

Applying code-based migration: 201110180052097_GallerySettings.

Applying code-based migration: 201110230649210_
PackageOwnerRequests.

...
```

Now, we're ready to run our NuGet Gallery. Navigate to **Debug | Start Debugging** or press *F5* to compile and start the application using IIS Express.

The browser should start and display a screenshot similar to the following:



Success! We have created our own private NuGet Gallery. Now, it's time for configuration:

1. First, we're going to get rid of the yellow banner on the top. Simply remove or rename the `Alert.md` file from the `App_Data\Files\Content` folder and the banner goes away.

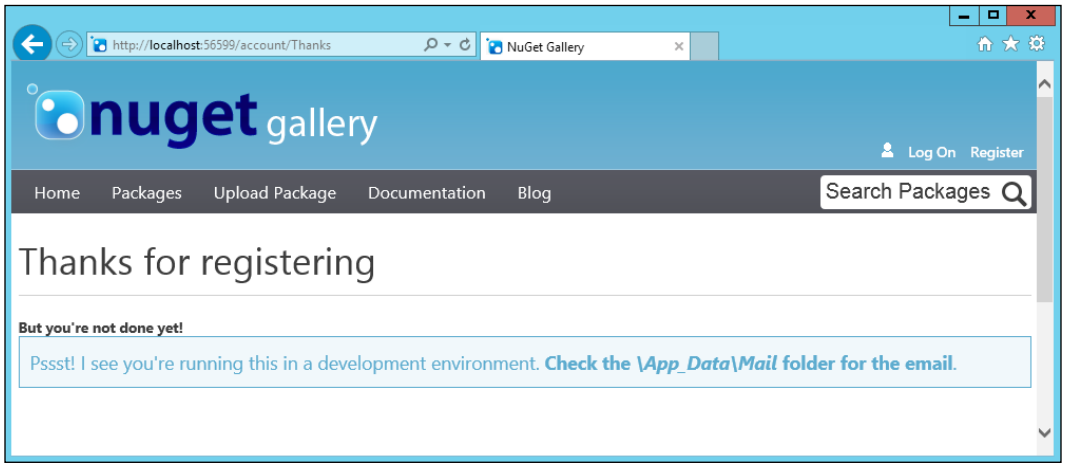
   If you ever need a custom alert message, you can use the `Alert.md` file to add this message. For example, with the following content in the `Alert.md` file:

   ```
   <div class="banner-warning">This is my message.</div>
   ```

   `This is my message.` will be displayed in a yellow banner.

2. We're going to create our user in the NuGet Gallery. Click on **Register** in the top-right side of the site. **Create A New Account** is shown with four textboxes. Fill these fields with your information and after agreeing with the terms, click on **Register**.

   We're redirected to the next page.



3. Let's activate our user. Navigate to the `App_Data\Mail` folder, in our case this is `C:\NuGetGallery\Website\App_Data\Mail`. The folder should contain files with an EML extension. Open a file with a text editor or mail client, find the verification link, and copy/paste it into your browser.

   [If you're using a text editor for EML files, beware of the = sign for delimiting a new line.]

When the page loads, our account is confirmed, suggesting us to click on **Log On** and log on to the site. Now we can log in to our NuGet Gallery. Note that we are not a super user right away, we have only basic permissions. To gain administrator permissions, we need to run a SQL query on our database.

4. Open **Microsoft SQL Server Management Studio** and connect to the database in our connection string; if you're using default settings, **Server name** should be `(LocalDB)\v11.0`. Start a new query by selecting the menu item by navigating to **File | New | Query with Current Configuration** and select our database from the databases drop-down list.

```
INSERT INTO [dbo].UserRoles VALUES ( (SELECT TOP 1 [Key]
  FROM [dbo].[Users]), 1)
```

The preceding SQL query will retrieve the user key from the database and then add a new `RoleKey` to this user. Since we are the only user in the database, a subselect will return our user's key and give us administrator permissions.

With the admin role, we can access the administration mode of our NuGet Gallery at `http://localhost:56599/admin`. There we have full access to the database, logs, and all configuration settings.

That's it; we have set up our own NuGet Gallery. Congratulations!

Now, we can look up our API key as described in *Chapter 3*, *Creating and Publishing a Package*, and upload packages using the following command:

```
C:\>nuget push GoogleSearchApi.1.0.0.nupkg -Source
  http://localhost:56599/api/v2/package
  -ApiKey 390478eb-5352-4dae-8237-92e5780b4eca

Pushing GoogleSearchApi 1.0.0 to
  'http://localhost:56599/api/v2/package'...

Your package was pushed.
```

Under the **Packages** tab in our NuGet Gallery, we can see that our package is listed correctly.

NuGet server in a continuous integration system

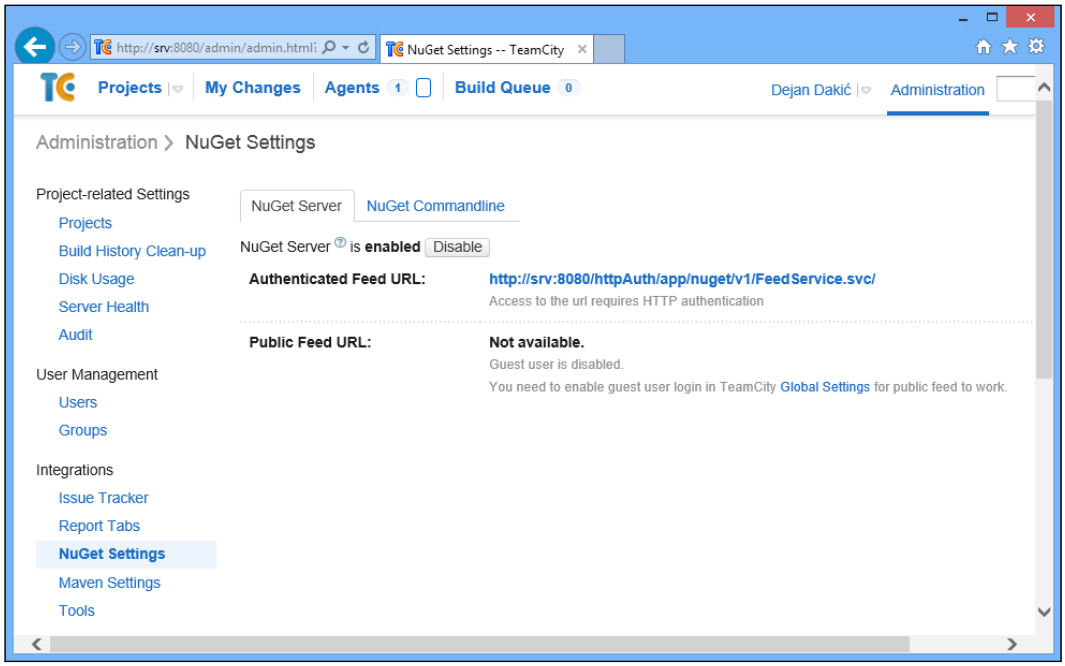
A common practice in enterprise environments is using continuous integration systems and build systems such as TeamCity and CruiseControl.NET. We can easily integrate our NuGet server with our build server, for example, to create a package and publish it in the internal server upon each successful build.

Some build systems go even one step further; for example, in TeamCity you can create your own NuGet server with just a few clicks. In the **Administration** dashboard, select **NuGet Settings** from the left pane and click on the **Enable** button. NuGet server is automatically created and configured.

This URL feed is unavailable for public use, as you first need to enable the guest user login by navigating to **Administration | Global Settings** in your TeamCity dashboard.

With build steps, you can easily configure packing and publishing of packages on this private or any other feed. Triggering a build when a NuGet package is updated is a typical scenario while performing test runs.

> TeamCity is a continuous integration system free for 20 build configurations. More information is available at `http://www.jetbrains.com/teamcity/`.



Summary

This chapter covered the basics of hosting an internal NuGet server. We started with a simple filesystem-based repository, which is acceptable for individual use, and then progressed to the more advanced alternatives. Advanced, but still very lightweight, we covered the `NuGet.Server` package. Using this solution allows us to handle security and authentication, but it still lacks statistics and unique users' API keys. We continued with the open source project NuGet Gallery, which offers us the same experience as the official NuGet feed. We covered the installation and configuration of the NuGet Gallery. We concluded the chapter with a discussion about the NuGet server available in one of the continuous integration systems.

Index

A

apiKey 82
AssemblyCompanyAttribute 44
AssemblyCopyrightAttribute 44
AssemblyDescriptionAttribute 44
AssemblyTitleAttribute 44
assembly version
  selecting 23, 24
authors element 38
Automatic Package Restore 29

B

build folder 49

C

Command-Line Package Restore 30
command-line utility
  NuGet, installing from 14
configuration files transformation
  about 60-62
  XDT syntax 62
  XML-Document-Transform syntax 62
Console Application template 57
content folder 49
continuous integration systems
  NuGet server 89, 90
copyright element 38

D

dependencies element 39
description element 38
Development Tools Environment (DTE) 68
dynamic keyword 12

E

Enable button 90
Extension Manager
  NuGet, installing from 13, 14

F

files
  configuration files, transforming 60-62
  source code files, adding 63, 64
filesystem-based NuGet repository
  creating 79
  NuGet cache 80
frameworkAssemblies element 39

I

iconUrl element 38
id element 38, 58
Internet Information Services (IIS) 80

J

Json.NET 15

L

language element 39
lib folder 49
licenseUrl element 38
Lightweight Directory Access Protocol (LDAP) 78
lightweight NuGet server
  about 80
  configuring 81
  creating 81

packages, accessing 83
private feed, using with package restore 83, 84

M

MAJOR.MINOR.PATCH 47
Manage NuGet Packages dialog box 59
minClientVersion element 39
MSBuild-Integrated Package Restore 29
multiplatform support
about 50, 51
folder naming convention 54
folder structure 51, 52
NuGet package, testing, locally 52, 53
MyDemoPackageCmd.nuspec command 43

N

NuGet
about 7
dependencies, handling 15, 16
installing, from command-line utility 14
installing, from Extension Manager 13, 14
installing, from Visual Studio gallery 13
settingup, in Visual Studio 12, 13
used, for adding references to third-party libraries 14, 15
using, on build server 30
using, with source control 26, 27
NuGet Gallery
about 84
compiling 86-89
configuring 85-89
features 85
installing 85
internal hosting, reasons 78
package, publishing to 41, 42
prerequisites 85
URL 77
NuGet package 49
NuGet Package Explorer
used, for package creating 35-40
NuGet Package Manager 49
NuGet package repository
creating 78
filesystem-based NuGet repository, creating 79
lightweight NuGet server, creating 80, 81
NuGet Gallery 84
NuGet server
in continuous integration systems 89, 90
nuget spec -a assembly.dll command 42
NuGet, Visual Studio
NuGet, installing 13
requirements 12, 13
setting up 12, 13
nuspec file 58
NuSpec reference 39

O

Online option 15

P

package
creating, dependencies 40
creating, from command line 42, 43
creating, NuGet Package Explorer used 35-39
debugging 70
generating, during build process 44-46
NuGet specification, creating 42
packing 43
publishing 43
publishing, from command line 42, 43
publishing, to NuGet Gallery 41, 42
symbol package, consuming 72-74
symbol package, creating 70, 71
symbol package, publishing 70
package folder structure
about 49, 50
build folder 49
content folder 49
lib folder 49
tools folder 49
Package Management Console window 69
package manager
using, benefits 7, 8
Package Manager Console
about 31
using 31-33
packagePath 82
Package Restore 26

package versioning
prerelease packages 47
SemVer 47
PowerShell scripts
including, in packages 67, 68
Init.ps1 67
Install.ps1 67
PowerShell module, adding 69
running, at install time 68
Uninstall.ps1 67
prerelease packages 47, 48
projectUrl element 38

R

reference third-party libraries
installed package, updating 20-22
package versions, managing 22
updating 19, 20
requireApiKey 82
requireLicenceAcceptance element 38

S

satellite assemblies
adding, to main package 56-58
packaging 55
satellite packages, advantage 59
satellite packages, creating 58, 59
Semantic Versioning. *See* **SemVer**
SemVer 47
Silverlight Class Library template 53
Solution Explorer window 21
source code files
adding 63
MSBuild properties, adding 65, 66
MSBuild targets, adding 65, 66
Visual Studio project properties 64
source control
about 26
NuGet, using with 26-30
summary element 38
symbol package
consuming 72-74
creating 70, 71
publishing 70, 71
symbols.nupkg file 71

T

tags element 38
TeamCity 90
Team Foundation Service (TFS) 26
third-party library references
adding, manually 8-12
title element 38
tools folder 49

U

unsupported target platform
package, installing for 25, 26

V

version element 38
Visual Studio gallery
NuGet, installing from 13

X

xdt:Locator attribute 62
XDT syntax
about 62
transform file 63
using 62
xdt:Transform attribute 63
XML-Document-Transform. *See* **XDT syntax**
XML element
authors 38
copyright 38
dependencies 39
description 38
frameworkAssemblies 39
iconUrl 38
id 38
language 39
licenseUrl 38
minClientVersion 39
owners 38
projectUrl 38
requireLicenceAcceptance 38
summary 38
tags 38
title 38
version 38

Thank you for buying

NuGet 2 Essentials

About Packt Publishing

Packt, pronounced 'packed', published its first book "*Mastering phpMyAdmin for Effective MySQL Management*" in April 2004 and subsequently continued to specialize in publishing highly focused books on specific technologies and solutions.

Our books and publications share the experiences of your fellow IT professionals in adapting and customizing today's systems, applications, and frameworks. Our solution based books give you the knowledge and power to customize the software and technologies you're using to get the job done. Packt books are more specific and less general than the IT books you have seen in the past. Our unique business model allows us to bring you more focused information, giving you more of what you need to know, and less of what you don't.

Packt is a modern, yet unique publishing company, which focuses on producing quality, cutting-edge books for communities of developers, administrators, and newbies alike. For more information, please visit our website: `www.packtpub.com`.

About Packt Open Source

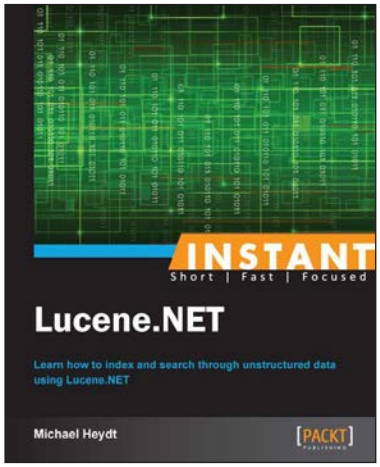
In 2010, Packt launched two new brands, Packt Open Source and Packt Enterprise, in order to continue its focus on specialization. This book is part of the Packt Open Source brand, home to books published on software built around Open Source licences, and offering information to anybody from advanced developers to budding web designers. The Open Source brand also runs Packt's Open Source Royalty Scheme, by which Packt gives a royalty to each Open Source project about whose software a book is sold.

Writing for Packt

We welcome all inquiries from people who are interested in authoring. Book proposals should be sent to author@packtpub.com. If your book idea is still at an early stage and you would like to discuss it first before writing a formal book proposal, contact us; one of our commissioning editors will get in touch with you.

We're not just looking for published authors; if you have strong technical skills but no writing experience, our experienced editors can help you develop a writing career, or simply get some additional reward for your expertise.

[PACKT] open source
community experience distilled
PUBLISHING

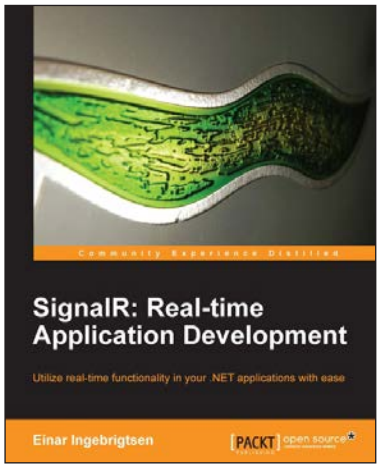


Instant Lucene.NET

ISBN: 978-1-78216-594-1 Paperback: 66 pages

Learn how to index and search through unstructured data using Lucene.NET

1. Learn something new in an Instant! A short, fast, focused guide delivering immediate results
2. Learn how to execute searches for document indexes
3. Understand scoring and influencing search results
4. Easily maintain your index



SignalR: Real-time Application Development

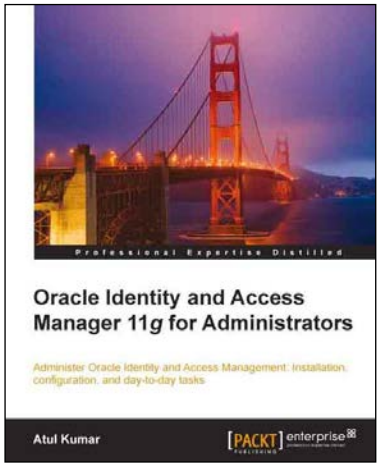
ISBN: 978-1-78216-424-1 Paperback: 124 pages

Utilize real-time functionality in your .NET applications with ease

1. Develop real-time applications across numerous platforms
2. Create scalable applications that are ready for cloud deployment
3. Utilize the full potential of SignalR

Please check **www.PacktPub.com** for information on our titles

[PACKT] open source
community experience distilled
PUBLISHING



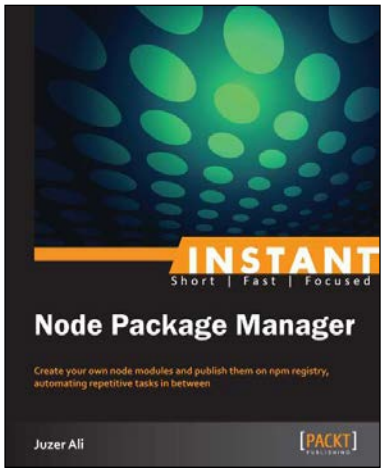
Oracle Identity and Access Manager 11*g* for Administrators

ISBN: 978-1-84968-268-8 Paperback: 336 pages

Administer Oracle Identity and Access Management: Installation configuration, and day-to-day tasks

1. Full of illustrations, diagrams, and tips with clear step-by-step instructions and real time examples
2. Understand how to Integrate OIM/OAM with E-Business Suite, Webcenter, Oracle Internet Directory and Active Directory
3. Learn various techniques for implementing and managing OIM/OAM with illustrative screenshots

---



Instant Node Package Manager

ISBN: 978-1-78328-333-0 Paperback: 56 pages

Create your own node modules and publish them on npm registry, automating repetitive tasks in between

1. Learn something new in an Instant! A short, fast, focused guide delivering immediate results
2. Create and distribute node modules
3. Learn how to publish executables
4. Automate the installation of dependencies

Please check **www.PacktPub.com** for information on our titles

www.allitebooks.com